Manual da SOLAR

Systematic Orbit-based Layered Architecture

Este manual explica **o que cada peça da SOLAR faz, por que existe e como se usa** — com exemplos, casos de uso e as violações que a arquitetura barra.

Está organizado em quatro blocos:

Bloco	Para quem	O que responde
1. Conceitos	quem está chegando	o que é a SOLAR e quais são as suas estruturas
2. Nucleus	quem vai escrever código	cada classe e cada método, com exemplo
3. Regras	quem vai revisar código	as 31 regras, violação e correção
4. Receitas e extensão	quem vai construir	casos de uso completos e como estender

> Todo o código deste manual é verificado pela própria SOLAR: o comando `python observatory/solar check` roda as 31 regras sobre o projeto e reprova o que estiver fora do lugar.

Sumário

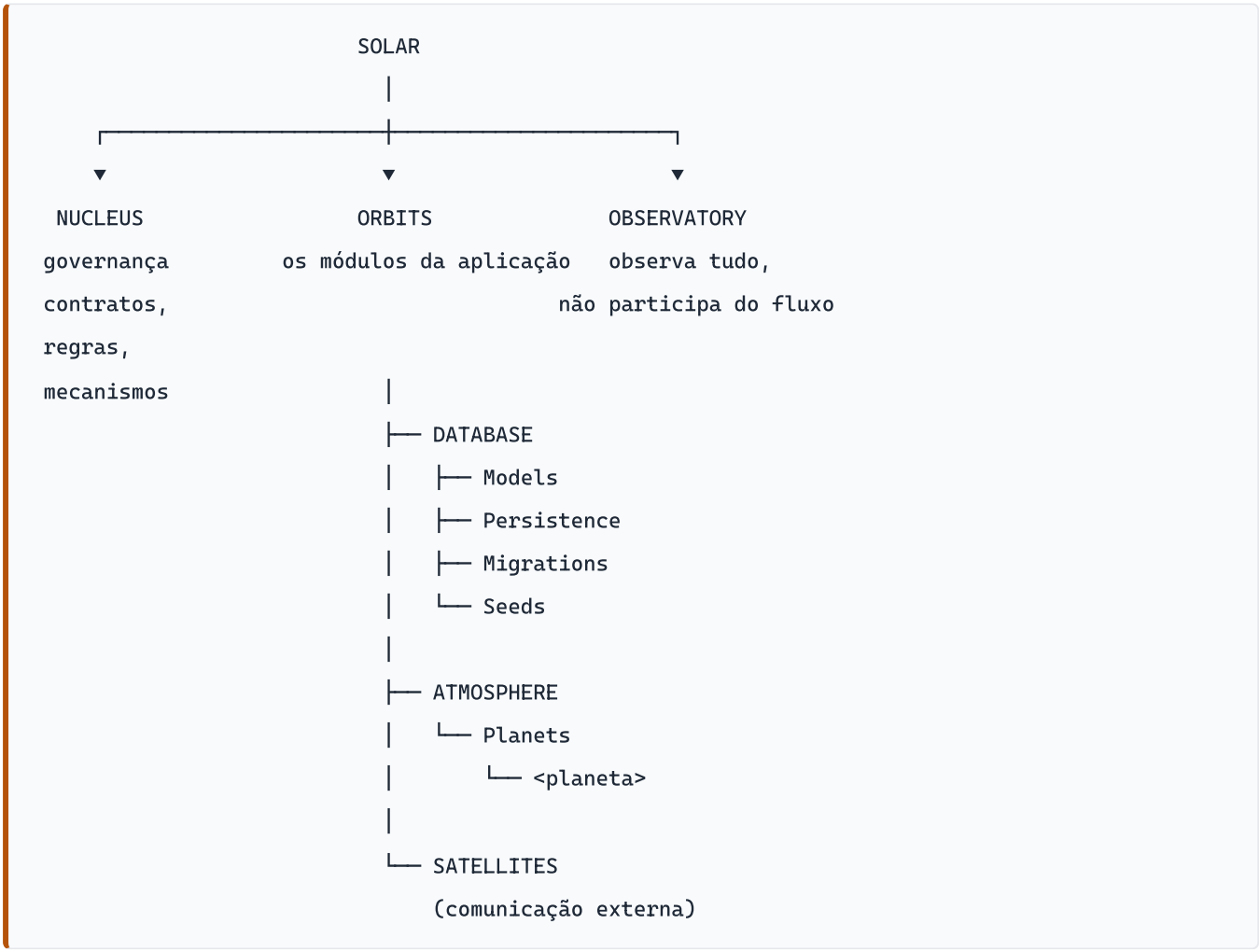
1. O que é a SOLAR
2. Começando
3. Nucleus — Exceções
4. Nucleus — Contratos
5. Nucleus — Middleware
6. Nucleus — Segurança
7. Nucleus — Regras
8. Nucleus — Banco de dados
9. Nucleus — Storage
10. Nucleus — Upload
11. Nucleus — Broadcast
12. Nucleus — Notificação
13. Nucleus — Environment
14. As 31 regras arquiteturais

15. Casos de uso
16. Estendendo a SOLAR
17. Verificação e auditoria
18. Índice de classes

---

1. O que é a SOLAR

A SOLAR organiza uma aplicação em três estruturas — **Nucleus**, **Orbits** e **Observatory** — e cada uma tem **uma responsabilidade** e **uma direção de dependência**.



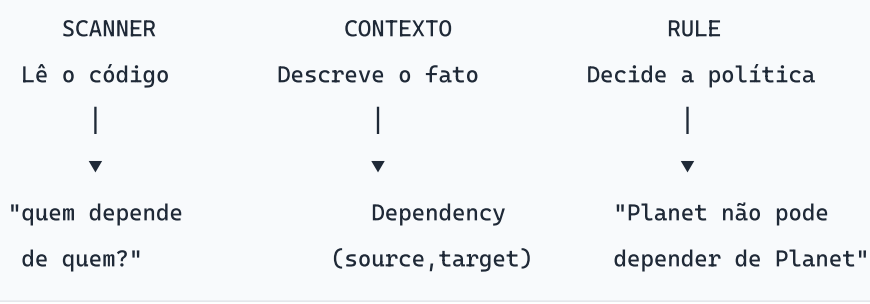
Estrutura	Papel	Onde vive
Nucleus	governança: contratos, regras, mecanismos	`nucleus/`
Orbits	os módulos da aplicação	`orbits/`
Database	persistência: Models, Persistence, Migrations e Seeds	`orbits/database/`
Atmosphere	a fronteira de entrada de cada Planet	`planets/<p>/atmosphere/`
Planets	os módulos de negócio	`orbits/planets/`
Satellites	fronteira de comunicação externa e entre módulos	`orbits/satellites/`
Observatory	observação: métricas, erros, verificação e auditoria	`observatory/`

> O **Environment não é uma camada**: é um **mecanismo do Nucleus** (`nucleus/environment.py`) — carrega a configuração, e só isso. Ver seção 13.

1.1 A ideia central

A SOLAR não é um conjunto de boas intenções escritas em um documento. É um conjunto de **regras que rodam**.

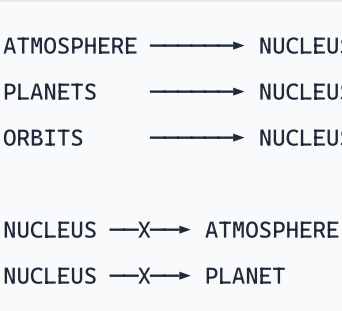
Uma regra arquitetural que ninguém executa vira folclore: todos concordam com ela e ninguém a cumpre. Por isso cada regra da SOLAR tem três partes que se completam:



- O **Scanner** percorre o código e **descreve fatos** — nunca decide o que é violação.
- O **Contexto** é o fato, em um `dataclass` imutável.
- A **Rule** olha o fato e diz se ele respeita a política.

Essa separação é o que permite testar a política sem tocar no código analisado, e é o que mantém `scanner` e `rule` independentes.

1.2 A direção das dependências



O Nucleus define as regras e **não sabe quem as usa**. O Observatory verifica e **não é conhecido pelo Nucleus**. Essa direção única é o que permite trocar a observação sem tocar na governança — e é ela que sustenta as regras de dependência (SOLAR-ARCH-008 a 012).

---

2. Começando

2.1 Estrutura

```
SOLAR/
├── nucleus/          governança: contratos, regras, mecanismos
├── orbits/           os módulos da aplicação
│   ├── planets/      os Planets (inclusive o `solar`, da própria plataforma)
│   │   └── <p>/atmosphere/   a fronteira de entrada do Planet
│   └── database/
│       ├── models/       os Models
│       ├── persistence/  o acesso a dados
│       ├── migrations/   as migrations
│       └── seeds/        os dados iniciais
├── observatory/      observação: métricas, erros e auditoria
│   ├── records/      o registro de erros (banco SQLite próprio)
│   └── solar/
│       └── check/    as auditorias geradas
├── tests/            testes (NÃO são verificados pelas regras)
├── docs/             documentação
└── docker/           infraestrutura — NÃO faz parte da arquitetura
```

> A pasta `docker/` é **infraestrutura**: fica fora da arquitetura SOLAR. Nenhum scanner olha para ela — ela não é uma camada, é o que coloca a aplicação de pé.

2.2 Comandos

```
# Testes
pytest -q

# Verificação arquitetural (o "portão")
python observatory/solar check
python observatory/solar check <caminho>

# Somente o que difere da última vez
git diff --stat observatory/solar/check/
```

2.3 O ciclo

```
escrever código
       |
       ▼
python observatory/solar check ──→ correct ──→ commit
       |
       └──→ violation ──→ observatory/solar/check/check-....md
                                  |
                                  ▼
                            corrigir e repetir
```

Cada execução **gera um registro**, com `correct` ou `violation`. O nome do arquivo carrega momento e resultado:

```
check-2026-10-05-155458-violation.md
check-2026-10-05-162011-correct.md
```

Então o arquivo mais recente — o último em ordem alfabética — diz, sem precisar abrir, se a última verificação passou.

2.4 Subindo a aplicação

Duas peças, com papéis separados:

```
Kernel   (nucleus/kernel.py)   MONTA: configuração, mecanismos, rotas, observação
Server   (server.py)           SOBE: recebe a aplicação montada e serve
```

```
python server.py
```

O `Server` é o ponto de entrada. Ele pede a montagem ao Kernel e sobe — e é ele que liga o Observatório. Isso não é escolha de gosto: a direção da dependência é Observatory → Nucleus, o Nucleus **não** conhece o Observatório. Quem conhece os dois é quem está fora de ambos.

Antes de montar, o `Server` **verifica a arquitetura** e mostra o resultado — pelo mesmo motivo: quem conhece os dois pode fazer isso, e o Kernel não. A verificação da subida registra auditoria **só quando há violação** (§17.8).

```
$ python server.py
verificando a arquitetura...
arquitetura: correct
 * Serving Flask app 'solar'
 * Running on http://localhost:5000
```

O que o Kernel monta, e em que ordem

A ordem da montagem é a ordem das dependências:

Etapa	O que liga
`Environment`	lê o `.env` pelo ambiente ativo (`{APP_ENV}_{CHAVE}`)
`Identity`	quem a aplicação é: `SECRET_KEY` e `SOLAR_ID`
Drivers	`DriverSQLAlchemy` , `LocalStorageDriver` , `InMemoryBroadcast`
Middleware	`SecurityPolicy` , `RequestPipeline` , `ErrorHandler`
Rotas	o ponto central `orbits/routes.py` , que agrega os Planets

A aplicação fala **sempre com as fachadas** (`Database` , `Storage` , `Broadcast`); qual driver está atrás delas é decisão de montagem, e é tomada em um lugar só.

O corpo de `mount()` **é** a receita da montagem: ler aquele bloco de cima para baixo é ler o que acontece quando a aplicação sobe. Não há um método privado por etapa — a montagem se lê de uma vez.

As dependências

Pacote	Para quê
`flask`	o framework: `Blueprint` + `add_url_rule`
`python-dotenv`	leitura do `.env`
`sqlalchemy`	a ORM — usada **só** pelo driver
`alembic`	comparar o banco real com os Models — só o driver usa
`psycopg[binary]`	o driver do Postgres
`bcrypt`	o hash de senha, usado pelo `Password`

O banco

O `docker-compose.yml` sobe o Postgres junto. O serviço tem `healthcheck` , e a aplicação só sobe depois que o banco responde — o `depends_on` com `service_healthy` existe para isso.

Ambiente	Banco
DEV	Postgres, no compose
TEST	`sqlite` em `:memory:`
PROD	Postgres

TEST não depende de nada externo: é o que permite rodar a suíte inteira sem subir banco.

O schema

O banco inicia montado. `Database.init()` conecta e, em seguida, monta as estruturas: cria as tabelas que faltam, altera o que mudou e remove o que sobrou. Banco conectado sem as tabelas certas é falha adiada para a primeira consulta.

```
Database.init(DriverSQLAlchemy(config))
```

É isso que a montagem da aplicação escreve — **uma linha, e nada sobre Model, tabela ou schema**. Quem cuida do próprio esquema é o banco (§8.1).

O caminho, de dentro para fora:

Passo	Quem	O quê
1	`Database.init`	valida o driver, publica `orm` / `model` e conecta
2	`Database._import_models`	importa os Models do pacote — importar é o que REGISTRA a tabela
3	`Database.sync` → driver	cria, altera e remove, até o banco ficar igual aos Models

A ordem é a da dependência, e é **obrigatória**: `class Users(Database.orm)` roda no import do Model, então `orm` precisa estar publicado antes — é por isso que o passo 2 só pode existir depois do passo 1.

Os Models são achados pelo **pacote** (`Database.MODELS_PACKAGE`): um Model novo é um arquivo novo, e nada mais.

O botão

```
class Database:
    SYNC_ON_START = True
```

Desligar é trocar para `False` : o banco conecta e não encosta no esquema.

Em produção isto deve ficar DESLIGADO. Sincronizar esquema cria, altera e **remove** estrutura — e um engano aqui custa dado. Em produção o esquema pertence ao **deploy**, não à subida da aplicação.

É também o ponto de troca para quem quiser migrations: com `SYNC_ON_START = False`, quem cuida do esquema passa a ser a ferramenta de migração, rodada antes de subir. O `sync` continua disponível para quem quiser chamá-lo na mão.

Isto **não** substitui migrations, e a diferença importa: sincronizar não é **versionar**. Não há revisão, não há histórico, não há volta atrás (§8.3).

---

3. Nucleus — Exceções

`nucleus/exceptions/`

Toda falha da arquitetura nasce de uma exceção da SOLAR. Não se levanta `Exception` genérica para um erro de negócio: quem captura precisa conseguir decidir o que fazer.

3.1 A hierarquia

```
Exception
└── SolarException                base de tudo
    ├── ContractException         um componente não respeita um contrato
    ├── DatabaseException         falha de persistência ou conexão
    ├── SecurityException         falha de segurança
    ├── StorageException          falha de armazenamento de arquivo
    ├── BroadcastException        falha ao publicar/ouvir evento
    └── ValidationException       a entrada não passou na validação
```

3.2 `SolarException`

```
class SolarException(Exception):
    def __init__(self, message: str, code: str | None = None)
    def __str__(self)
```

`message`	o que aconteceu, em texto para quem lê
`code`	identificador estável, em `UPPER_CASE`

Por que o `code` existe. A mensagem pode mudar de redação; o código não. É ele que sobrevive à tradução, ao log e ao tratamento automático:

```
raise ValidationException(
    "O arquivo excede 10 MB.",
    code="FILE_TOO_LARGE"
)
```

E é ele que o `ErrorHandler` devolve ao cliente — sem expor mais nada:

```
str(SolarException("arquivo inválido", code="INVALID_UPLOAD"))
# → "[INVALID_UPLOAD] arquivo inválido"
```

3.3 Quando usar cada uma

Exceção	Use quando	Exemplo de `code`
`ContractException`	um objeto recebido não implementa o contrato esperado	`INVALID_DATABASE_DRIVER`
`ValidationException`	a entrada do usuário está errada	`INVALID_UPLOAD`
`DatabaseException`	a persistência falhou ou não foi inicializada	`DATABASE_NOT_INITIALIZED`
`StorageException`	falha ao gravar/ler arquivo	`STORAGE_PATH_ESCAPE`
`SecurityException`	URL barrada, token CSRF inválido	`INVALID_CSRF_TOKEN`
`BroadcastException`	falha ao publicar evento	`BROADCAST_PUBLISH_ERROR`

3.4 Violação e correção

```
# [X] mensagem interna na exceção que sobe para o cliente
except Exception as error:
    raise BroadcastException(f"Falha ao publicar: {error}", code="PUBLISH_ERROR")

# [OK] mensagem genérica, código estável, causa preservada para o log
except Exception as error:
    raise BroadcastException("Falha ao publicar evento.", code="PUBLISH_ERROR") from error
```

O `from error` **preserva a causa**: o log mostra a cadeia completa, o cliente vê só o código. Essa é a SOLAR-ARCH-028 em uma linha.

---

4. Nucleus — Contratos

`nucleus/contracts/`

Contratos são as fronteiras do Nucleus. Eles dizem **qual é a forma** de um componente, sem dizer quem o implementa — é isso que permite ao Planet ignorar se a persistência é SQLite ou Postgres.

> Todos os contratos usam `ABC`. Um objeto que não implementa o contrato não é aceito: os mecanismos verificam com `isinstance` e levantam `ContractException`.

4.1 `Validator`

```
class Validator(ABC):
    def validate(self, payload) -> None
```

Valida a entrada. **Não retorna nada**: ou passa em silêncio, ou levanta `ValidationException`.

```
class DocumentValidator(Validator):

    def validate(self, payload) -> None:

        errors = []

        if not payload.get("title"):
            errors.append("O título é obrigatório.")

        if errors:
            raise ValidationException(
                " ".join(errors),
                code="INVALID_DOCUMENT"
            )
```

A assinatura não tem tipo no `payload` de propósito: o mesmo contrato serve a JSON, form e multipart. Quem sabe o tipo é o validator concreto.

4.2 `Sanitizer`

```
class Sanitizer(ABC):
    def sanitize(self, payload)
```

Limpa a entrada e **devolve a versão limpa**. Roda **antes** do `Validator` — validar dado sujo é validar a coisa errada.

```
class TextSanitizer(Sanitizer):


    def sanitize(self, payload):
        return {
            key: value.strip() if isinstance(value, str) else value
            for key, value in payload.items()
        }
```

4.3 `Resource`

```
class Resource(ABC):
    @classmethod
    def to_object(cls, data) -> dict
    @classmethod
    def collection(cls, data) -> list
```

Transforma o resultado do UseCase no que vai para a resposta. É a fronteira de saída: o UseCase devolve entidades, o Resource decide o que é público.

O Resource é **chamado, e não instanciado** — `to_object` é `classmethod`, porque não há estado a guardar entre a entrada e a saída:

```
class DocumentResource(Resource):


    @classmethod
    def to_object(cls, data) -> dict:
        return {
            "id": data.id,
            "title": data.title,
            "created_at": data.created_at.isoformat(),
        }
```

O Controller chama e devolve o resultado:

```
return DocumentResource.to_object(output), 201
```

`collection` existe para o caso da lista: `[DocumentResource.to_object(d) for d in docs]` é a mesma ideia repetida em todo Controller, e a repetição é onde um item acaba escapando sem passar pelo Resource.

É aqui que a SOLAR-ARCH-021 age: se o dicionário expuser `password`, `password_hash`, `secret_key` ou um token, a regra reprova.

4.4 `Controller`

Um Controller **não tem contrato de classe**. Não há base a herdar, decorator a aplicar nem registro a fazer: é uma classe simples, reconhecida pelo **lugar** onde foi declarada (`planets/<planeta>/controllers/`, §7.8) e pelo sufixo `Controller` no nome.

```
class CreateUserController:

    @staticmethod
    def create() -> tuple[dict, int]:
        ...
```

`@staticmethod` porque a rota é registrada como `CreateUserController.create`: o Flask chama a função direto, sem instância no meio. Quem exige que o retorno passe por um Resource é a SOLAR-ARCH-014, na análise estática (§14.6) — e não um contrato de classe.

4.5 `UseCase`

```
class UseCase(ABC):
    def execute(self, request)
```

Executa **uma** responsabilidade. É onde mora a regra de negócio — e é o último lugar onde a decisão de negócio pode existir.

4.6 `ResponsibilityContract`

```
@dataclass(frozen=True)
class ResponsibilityContract:
    owner: str
    input_type: object
    output_type: object
```

Declara **o que entra e o que sai** de um UseCase. É o contexto da SOLAR-ARCH-015.

```
ResponsibilityContract(
    owner="CreateDocumentUseCase",
    input_type=DocumentInput,      # [OK] nomeado
    output_type=DocumentOutput,    # [OK] nomeado
)
```

```
ResponsibilityContract(
    owner="CreateDocumentUseCase",
    input_type=dict,               # [X] built-in literal
    output_type=dict,
)
```

Contrato nomeado é o que permite evoluir a fronteira sem quebrar quem chama. Um `dict` anônimo não tem onde receber um campo novo sem virar combinado verbal.

4.7 `HttpInput`

```
@dataclass(frozen=True)
class HttpInput:
    method: str
    name: str
    content_type: str | None = None
    sanitizer: Sanitizer | str | None = None
    validator: Validator | str | None = None
    carries_data: bool | None = None

    def transports_data(self) -> bool
```

Descreve **uma entrada HTTP**. É o contexto da SOLAR-ARCH-013 (sanitização e validação).

Campo	Papel
`method`	o verbo (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`)
`name`	a rota (`/documents`)
`content_type`	`application/json`, `multipart/form-data`, …
`sanitizer` / `validator`	a instância em execução, ou o **nome** na verificação estática
`carries_data`	se transporta dados externos (ver abaixo)

`transports_data()` — o critério não é o verbo

```
def transports_data(self) -> bool:

    if self.carries_data is not None:
        return self.carries_data

    return self.content_type is not None
```

O que obriga sanitização e validação é **transportar dados**, não ser `POST`:

Entrada	Transporta?	Por quê
`GET /solar/health`	não	não recebe corpo
`POST /documents` (multipart)	sim	recebe arquivo
`PUT /users` (json)	sim	recebe corpo
`POST /search` (sem `content_type`)	**sim**	o método aceita corpo

A última linha é o ponto: o scanner marca `carries_data=True` para rota cujo método aceita corpo **mesmo sem `content_type` declarado**. Sem isso, bastaria omitir o tipo de conteúdo para escapar da regra.

> Declarar o tipo de conteúdo é **opcional**, e a rota não declara `sanitizer` nem `validator`: o que protege a entrada é a Atmosphere do Planet (§5.9), e o Controller recebe o resultado.

5. Nucleus — Middleware

`nucleus/middleware/`

Mecanismos que rodam **em volta** do caso de uso: antes (entrada), depois (saída) ou na fronteira (proteção).

5.1 `RequestPipeline`

```
class RequestPipeline:
    def handle(self, http_input: HttpInput, payload)
    def _sanitizer(self, http_input: HttpInput) -> Sanitizer
    def _validator(self, http_input: HttpInput) -> Validator
```

Garante, em **tempo de execução**, o que a SOLAR-ARCH-013 exige estaticamente.

```
payload = RequestPipeline().handle(http_input, request.json)
```

A ordem é sempre a mesma, e não é negociável:

```
   payload cru
        |
        ▼
   1) valida a regra 013   (a entrada declara sanitizer e validator?)
        |
        ▼
   2) valida               (os campos esperados existem?)
        |
        ▼
   3) sanitiza             (limpa os valores)
        |
        ▼
   payload confiável
```

Validar antes de sanitizar é a ordem: primeiro se confere se os campos esperados **existem** — se não existir o que limpar, não há o que o Sanitizer faça. Limpar o que não se sabe se está lá é limpar às cegas.

Só roda quando `transports_data()` é verdadeiro: uma rota que não recebe dados não tem o que validar nem o que sanitizar.

Um `sanitizer` que seja apenas o **nome** (como o scanner encontra no código estático) não serve aqui: o pipeline exige a instância e levanta `ContractException` com `INVALID_SANITIZER` / `INVALID_VALIDATOR`.

5.2 O Resource na resposta

Não há pipeline de saída. A resposta passa pelo Resource **no próprio Controller**, na forma de uma chamada a `to_object`:

```
return UserResource.to_object(output), 201
```

Quem garante isso não é um mecanismo em tempo de execução: é a **SOLAR-ARCH-014**, verificada na análise estática (§14.6). A regra lê o `return` de cada método público de Controller e exige que ele chame o Resource (`to_object` ou `collection`).

A troca foi deliberada. Um pipeline de saída só existiria para aplicar essa regra, e aplicar uma regra de arquitetura a cada resposta custa mais — e aparece menos — do que reprovar o código que não a segue.

5.3 `SecurityPolicy`

```
@dataclass(frozen=True)
class SecurityPolicy:
    content_security_policy: str | None = None
    strict_transport_security: str | None = None
    extra_headers: Mapping[str, str] = {}

    rate_limit_enabled: bool = False
    rate_limit_requests: int = 60
    rate_limit_window_seconds: int = 60

    brute_force_enabled: bool = False
    brute_force_max_attempts: int = 5
    brute_force_lock_seconds: int = 300

    csrf_enabled: bool = True
    csrf_ttl_seconds: int = 3600
```

Uma política, várias proteções. A separação entre o que é obrigatório e o que é opcional é o ponto:

Proteção	Padrão	Por quê
Cabeçalhos de segurança	**sempre ligados**	a proteção básica não pode depender de alguém lembrar
CSP / HSTS	desligados	dependem da aplicação
Rate limiting	desligado	depende do tráfego esperado
Brute force	desligado	depende do fluxo de login
CSRF	**ligado**	é defesa em profundidade, aplicada pelo pipeline

5.4 `SecurityHeaders`

```
class SecurityHeaders:
    MANDATORY = {
        "X-Content-Type-Options": "nosniff",
        "X-Frame-Options": "DENY",
        "Referrer-Policy": "no-referrer",
    }

    def headers(self) -> dict[str, str]
    def apply(self, headers=None) -> dict[str, str]
```

```
response.headers.update(SecurityHeaders(policy).headers())
```

Os três obrigatórios vão sempre. E `apply` faz a aplicação **não poder desligá-los**:

```
SecurityHeaders().apply({"X-Frame-Options": "ALLOWALL"})
# → {"X-Frame-Options": "DENY"}      o cabeçalho de segurança vence
```

5.5 `RateLimiter`

```
class RateLimiter:
    def __init__(self, policy=None, tracker=None)
    def allow(self, key: str) -> bool
    def retry_after(self, key: str) -> int
    def reset(self, key: str) -> None
```

Limita requisições por **chave**, dentro de uma janela.

```
if not limiter.allow(f"ip:{request.remote_addr}"):
    return jsonify({"error": "too_many_requests"}), 429, {
        "Retry-After": str(limiter.retry_after(key))
    }
```

A chave é sua escolha: `ip:...`, `user:42`, `route:/login`. Desligado na política, `allow()` devolve sempre `True` — não há caminho de exceção.

5.6 `BruteForceGuard`

```
class BruteForceGuard:
    def is_locked(self, key: str) -> bool
    def register_failure(self, key: str) -> None
    def register_success(self, key: str) -> None
    def retry_after(self, key: str) -> int
    def remaining_attempts(self, key: str) -> int
```

```
guard = BruteForceGuard(policy)

if guard.is_locked(f"email:{email}"):
    return 429

# ... autentica ...

guard.register_success(f"email:{email}")   # quem acertou é perdoado
```

Três decisões embutidas:

- **`register_success` zera o contador.** O guard protege contra tentativa e erro; quem acertou a senha não é punido pelas tentativas anteriores.
- **Bloqueio por chave**, não global: travar todos porque um e-mail errou é negar o serviço.
- **`remaining_attempts` existe** para a mensagem poder ser útil ("restam 2 tentativas") sem revelar se o e-mail existe.

5.7 `CsrfProtection`

```
class CsrfProtection:
    HEADER = "X-CSRF-Token"
    FIELD = "csrf_token"

    def __init__(self, ttl_seconds: int = 3600, clock=None, token_factory=None)
    def issue(self, session: str) -> str
    def token(self, session: str) -> str | None
    def is_valid(self, session: str, token: object) -> bool
    def validate(self, session: str, token: object) -> None
    def reset(self, session: str) -> None
```

```
csrf = CsrfProtection()

# ao renderizar o formulário
token = csrf.issue(session_id)

# antes do UseCase
csrf.validate(session_id, request.headers.get(csrf.HEADER))
```

Não se declara CSRF rota a rota. A proteção vive no pipeline: o backend emite o token e trava o cookie de sessão (`SameSite` , `HttpOnly` , `Secure`), e o navegador de outra origem não consegue usá-lo. Não há `csrf=True` para lembrar de escrever em cada rota.

Detalhe	Por quê
token **por sessão**	um token global não distingue quem é quem
TTL	um token vazado deixa de servir sozinho depois do prazo
`hmac.compare_digest`	comparação comum para no primeiro caractere diferente — e deixa medir o token caractere a caractere
`issue` **renova**	trocar o token a cada emissão limita a janela de uso

5.8 `ErrorHandler`

```
class ErrorHandler:
    GENERIC_MESSAGE = "Não foi possível processar a solicitação."
    INTERNAL_CODE = "INTERNAL_ERROR"

    def __init__(self, expose_details: bool = False, identify=None, observers=None)
    def handle(self, error: BaseException, context: Mapping | None = None) -> dict
    def code(self, error: BaseException) -> str
    def message(self, error: BaseException) -> str
    def reference(self, error: BaseException) -> str
    def detail(self, error: BaseException) -> str
```

Separa o que **sai** do que **fica**:

```
except Exception as error:
    logger.exception("falha ao processar o documento")   # detalhe → log
    return ErrorHandler().handle(error)                  # seguro → resposta
```

```
{
    "code": "INVALID_UPLOAD",
    "message": "Não foi possível processar a solicitação.",
    "reference": "a3f1c9d2e0b4"
}
```

Método	Serve para
`handle`	o payload que vai para quem chamou
`detail`	o texto que vai para o **log** (aqui o detalhe interno é bem-vindo)
`code`	o `code` da exceção, ou `INTERNAL_ERROR`
`reference`	liga a resposta ao log

A `reference` é o que torna a mensagem genérica **útil**: quem reporta o problema informa o código, e o time acha o detalhe no servidor — sem que ele tenha saído de lá. Ela fica guardada na própria exceção, então `handle()` e `detail()` devolvem sempre a mesma.

`expose_details=True` existe para desenvolvimento. Em produção é exatamente o vazamento que a SOLAR-ARCH-028 proíbe.

5.9 A fronteira do Planet (`atmosphere`)

`nucleus/middleware/atmosphere.py`

Uma peça, e a fronteira de entrada do Planet fica de pé:

Peça	O quê
`SolarBlueprint`	aceita `sanitizer=` / `validator=` na rota e liga `apply`
`apply(entries)`	devolve a função que o mecanismo liga em `before_request`

`apply(entries)`

```
def apply(entries):
    def protect():
        setattr(g, TREATED_DATA, {})
        guards = entries.get(local_endpoint())
        if guards is None:
            return None
        sanitizer, validator = guards
        payload = request.get_json(silent=True)
        setattr(g, TREATED_DATA, _PIPELINE.handle(
            HttpInput(
                method=request.method,
                name=request.path,
                sanitizer=sanitizer,
                validator=validator,
                carries_data=True,
            ),
            payload
        ))
        return None
    return protect
```

`entries` é o mapa que o **`SolarBlueprint`** monta a partir dos guardas que cada ROTA declarou (§15.6). O que a rota escreve:

```
# planets/example/routes.py
blueprint = SolarBlueprint("example", __name__, url_prefix="/examples")

blueprint.add_url_rule(
    "/users",
    "create_example_user",
    CreateUserController.create,
    methods=["POST"],
    sanitizer=CreateUserSanitizer,
    validator=CreateUserValidator,
)
```

É um `before_request` **de verdade**, e não um pipeline inventado: roda antes de qualquer Controller daquele Blueprint, e o comportamento é o mesmo das rotas do framework.

`local_endpoint`. Registrado num Blueprint, o Flask prefixa o endpoint com o nome dele: `/examples/users` vira `example.create_example_user`. O mapa usa o nome **local** — o que está escrito no `add_url_rule` —, e o último segmento é quem o entrega. Blueprint aninhado (`orbits.example.create_example_user`) também funciona.

Endpoint fora do mapa passa direto. É rota que não transporta dados (um GET, por exemplo), e não há o que validar nem o que limpar.

O corpo tratado espera no `g`. É o contexto da *requisição*: duas requisições simultâneas não se enxergam, o que um atributo de módulo não garantiria.

Como o Controller recebe o dado

O Controller lê `g.solar_data`:

```
from flask import g


class CreateUserController:

    @staticmethod
    def create() -> tuple[dict, int]:

        # O corpo JÁ TRATADO pela Atmosphere.
        data = g.solar_data

        # O corpo vira CONTRATO nomeado. Quem MONTA o contrato é o
        # módulo dele: o Controller só pede.
        user_input = create_user_input(data)

        output = create_user_use_case().execute(user_input)

        return UserResource.to_object(output), 201
```

`g` é o mecanismo do Flask para dado por **requisição**: a Atmosphere escreve, o Controller lê.

O Controller conhece o Flask, e isso é decisão consciente. A alternativa seria inventar um contrato de `Request` da SOLAR e um adaptador de view só para esconder o framework — e o custo de leitura não se pagava: quem chega no projeto já conhece o Flask, e a indireção teria que ser aprendida antes de qualquer linha útil.

O que chega vazio. `g.solar_data` é sempre um dicionário: rota sem guarda declarado recebe `{}`, e não um `AttributeError` que apareceria longe da causa. O sintoma é a AUSÊNCIA do dado, e não um dado cru — dado cru pareceria funcionar. Quem acusa é a SOLAR-ARCH-013, na análise estática (§14.2).

Uma instância por mecanismo. O pipeline é criado uma vez, na importação: ele não guarda estado entre requisições, e recriá-lo a cada chamada montaria um `RuleEngine` por requisição sem mudar o resultado.

---

6. Nucleus — Segurança

`nucleus/security/`

6.1 `Password`

```
class Password:
    def hash(password: str) -> str
    def verify(password: str, hashed_password: str) -> bool
```

```
hashed = Password.hash("minha-senha")
Password.verify("minha-senha", hashed)   # True
```

Usa **bcrypt**, com salt gerado por senha. O salt é o que impede que duas senhas iguais produzam o mesmo hash — sem ele, uma tabela pré-calculada quebra o banco inteiro.

É este componente que o Controller **deve** usar em vez de `hashlib` / `bcrypt` direto (SOLAR-ARCH-022): a decisão de como guardar senha é da arquitetura, não de cada Controller.

6.2 `Token`

```
class Token:
    def __init__(self, secret_key: str, expires_hours: int)
    def create(self, id_user: int) -> str
    def verify(self, token: str) -> dict
```

```
token = Token(secret_key=identity.secret_key, expires_hours=8)

jwt_string = token.create(id_user=42)
payload = token.verify(jwt_string)     # {"sub": "42", "exp": ...}
```

O que já está certo aqui, e por quê:

Prática	Efeito
`algorithm="HS256"` explícito	algoritmo declarado, nunca inferido do token
`algorithms=["HS256"]` na leitura	recusa token que diga ser de outro algoritmo
`"exp"` no payload	token **sempre** com prazo
`Bearer` removido	aceita o cabeçalho `Authorization` direto

Verificar um JWT sem declarar o algoritmo aceito é a falha clássica: o token diz qual é o algoritmo, e um token malicioso pode dizer "nenhum". É isso que a SOLAR-ARCH-027 barra.

> `verify` levanta `ValueError`, não `SolarException`. Quem trata deve converter: `raise SecurityException("Sessão expirada.", code="EXPIRED_SESSION") from error`.

6.3 `Identity`

```
class Identity:
    SECRET_KEY = "SECRET_KEY"
    SOLAR_ID = "SOLAR_ID"

    def __init__(self, env_file=".env")
    def init(self)
    def _load_environment(self)
    def _generate_secret_key()
    def _generate_solar_id()
    def _persist(self)
```

Gerencia a identidade criptográfica da aplicação: a `SECRET_KEY` (secreta) e o `SOLAR_ID` (não secreto, para logs e auditoria).

A `init` é **idempotente**: se os valores já existem no ambiente, são preservados; se não existem, são gerados e persistidos no `.env`.

```
identity = Identity()
identity.init()
```

> Repare: `SECRET_KEY = "SECRET_KEY"` é o **nome** da variável de ambiente, não um segredo. A SOLAR-ARCH-020 acusa essa linha porque o nome casa o vocabulário de segredo — é um falso positivo conhecido, ainda em aberto.

6.4 `AttemptTracker`

```
class AttemptTracker:
    def __init__(self, clock: Callable[[], float] | None = None)
    def now(self) -> float
    def record(self, key: str) -> None
    def count(self, key: str, window: float) -> int
    def oldest(self, key: str, window: float) -> float | None
    def reset(self, key: str) -> None
    def lock(self, key: str, seconds: float) -> None
    def locked_until(self, key: str) -> float | None
```

O **estado compartilhado** entre `RateLimiter` e `BruteForceGuard`: guarda tentativas por chave dentro de uma janela e administra bloqueios.

O `clock` injetável é o que torna o tempo testável:

```
relogio = FakeClock()

limiter = RateLimiter(SecurityPolicy(rate_limit_enabled=True), AttemptTracker(clock=relogio))

limiter.allow("ip:1.2.3.4")     # True
relogio.advance(61)             # o tempo passa...
limiter.allow("ip:1.2.3.4")     # True — a janela abriu
```

Tentar provar expiração com `time.sleep` tornaria o teste lento e instável.

O estado é **em memória, por processo**. Para várias instâncias da aplicação, troque o armazenamento mantendo o contrato — `count`, `oldest`, `lock` e `locked_until` são a interface que os dois mecanismos usam.

6.5 `UrlGuard`

```
class UrlGuard:
    DEFAULT_SCHEMES = ("https",)

    def __init__(self, allowed_hosts=None, allowed_schemes=None)
    def is_allowed(self, url) -> bool
    def validate(self, url) -> None
```

Valida uma URL **antes** de a aplicação buscar um serviço externo (SOLAR-ARCH-030).

```
guard = UrlGuard(allowed_hosts=["api.exemplo.com"])
guard.validate(url)                # levanta SecurityException se não puder

requests.get(url)
```

O que é barrado

Situação	Exemplo	`code`
esquema fora da lista	`file:///etc/passwd`	`DISALLOWED_SCHEME`
endereço interno	`https://169.254.169.254/latest/meta-data/`	`PRIVATE_ADDRESS`
	`https://127.0.0.1/`, `https://10.0.0.5/`, `https://[::1]/`	
host fora da lista	`https://outro.com/`	`DISALLOWED_HOST`
credenciais na URL	`https://user:senha@api.exemplo.com`	`URL_CREDENTIALS`
URL vazia ou inválida	`" "`, `None`, `"sem-esquema"`	`INVALID_URL`

Por que o metadata service importa. Em nuvem, `169.254.169.254` entrega credenciais da instância. Uma requisição que o atacante escolhe o destino é o suficiente para lê-las. É o alvo clássico de SSRF.

A limitação, dita de frente. O bloqueio de endereço interno vale para **IP literal**. Um nome que resolve para endereço interno só é barrado pela lista de hosts — resolver DNS aqui seria uma consulta de rede dentro do validador. Por isso `allowed_hosts` é a defesa principal.

6.6 `JsonValidator` e `JsonSanitizer`

```
class JsonSanitizer(Sanitizer):
    INVALID_PAYLOAD = "INVALID_JSON_BODY"
    def sanitize(self, payload) -> dict

class JsonValidator(Validator):
    INVALID_PAYLOAD = "INVALID_JSON_BODY"
    MISSING_FIELDS = "MISSING_FIELDS"
    def __init__(self, required=())
    def validate(self, payload) -> None
```

`nucleus/security/json_sanitizer.py` e `nucleus/security/json_validator.py`

Implementações prontas dos contratos `Sanitizer` e `Validator` (§4.1, §4.2), para o caso mais comum de todos: um corpo JSON que veio de fora. São dois guardas pequenos e genéricos — o Planet declara **quais**

campos são obrigatórios, e não reescreve a conferência.

`JsonSanitizer`

```
JsonSanitizer().sanitize({"name": "  Ana  ", "apelido": None})
# → {"name": "Ana"}
```

O que ele faz	Por quê
`strip()` no texto	`" Ana "` e `"Ana"` são o mesmo nome
descarta a chave `None`	ausência escrita de outra forma
deixa os outros tipos	`strip` em número ou booleano estouraria
devolve um mapa **novo**	quem chamou continua com o corpo que mandou

`JsonValidator`

```
JsonValidator(required=("name", "email")).validate({"name": "Ana"})
# ValidationException: [MISSING_FIELDS] O campo 'email' é obrigatório.
```

Um campo obrigatório está faltando em três casos, e os três **são o mesmo problema** para quem consome o dado:

Situação	Exemplo
a chave não veio	`{}`
a chave veio `None`	`{"name": None}`
a chave veio em branco	`{"name": " "}`

Separatórias dariam três mensagens para uma coisa só. E a recusa é **uma só**: o `Notification` (§12) acumula **todos** os campos que faltaram antes de levantar. Acusar um por vez obrigaria quem está corrigindo a chamada a uma rodada por campo.

O que nenhum dos dois faz

O escopo é **estrutura, não negócio**. Aqui se responde "o corpo tem os campos que a entrada exige?". "O e-mail tem formato válido?" é do UseCase, e "o usuário já existe?" é do banco.

Corpo que não é objeto — lista, texto solto, número — não passa por nenhum dos dois:

```
{
  "code": "INVALID_JSON_BODY",
  "message": "Não foi possível processar a solicitação.",
  "reference": "a3f1c9d2e0b4"
}
```

A mensagem que sai é a genérica do `ErrorHandler` (§5.8); **qual** campo faltou fica no log. É a SOLAR-ARCH-028 aplicada à entrada.

---

7. Nucleus — Regras

`nucleus/rules/`

É o coração da governança: como uma decisão arquitetural vira algo que roda.

7.1 `Rule`

```
class Rule(ABC):
    def code(self) -> str
    def description(self) -> str
    def validate(self, context) -> None
```

`code`	`"SOLAR-ARCH-019"` — o identificador
`description`	uma frase, para o relatório
`validate`	olha o contexto; **levanta** `SolarException` se houver violação

Ausência de exceção significa que está tudo certo. É o contrato inteiro: uma regra que "devolve False" obriga todo chamador a decidir o que fazer com isso; uma que levanta já diz.

7.2 `RuleEngine`

```
class RuleEngine:
    def __init__(self, rules: list[Rule])
    def validate(self, target) -> None
```

Roda uma lista de regras sobre **um** alvo.

```
engine = RuleEngine([PlanetCannotDependOnPlanet(), OrmBelongsToDriver()])
engine.validate(dependency)
```

Para na primeira violação — o contrato é "validar ou levantar". Para ver tudo de uma vez, quem responde é o `ProjectValidator` (ver 7.4).

> **Atenção:** cada contexto tem o seu engine. Passar um `HttpInput` para um engine de regras de `Dependency` quebra a execução: as regras esperam campos que não existem.

7.3 O padrão: Scanner → Contexto → Rule

```
código-fonte
     |
     ▼
 SCANNER              usa `ast`, NUNCA executa o código
     |                responde: "o que está escrito?"
     ▼
 CONTEXTO             dataclass frozen: o fato, sem julgamento
     |
     ▼
 RULE                 responde: "isso pode?"
     |
     ▼
 SolarException       ou silêncio
```

O `ArchitectureScanner`, por exemplo, não decide nada — ele só responde **quem depende de quem**:

```
source = "orbits.planets.user.repository"
target = "orbits.database.persistence.persistence"
```

Quem proíbe é a regra. Essa separação tem uma consequência prática: a mesma varredura alimenta **onze** regras (SOLAR-ARCH-001 a 012) sem custo extra.

O scanner lê, a regra julga

Vários contextos carregam **fatos ambíguos**, e a regra decide:

Contexto	Fato	Decisão da regra
`SecretOccurrence`	casou um padrão, está em docstring, tem marcador...	023 ignora documentação, exemplo e marcador
`ErrorExposure`	o detalhe sai, vai para o log, ou não se sabe	028 acusa só o que **sai**
`OutboundRequest`	quais identificadores compõem a URL	030 decide quais são confiáveis

O motivo é sempre o mesmo: o scanner não tem como saber o contexto de negócio — e não deve saber. Ele descreve; a política é uma decisão que pertence à regra.

7.4 `ProjectValidator`

```
@dataclass(frozen=True)
class Check:
    name: str
    scanner: type
    rules: tuple[Rule, ...]

def project_checks() -> tuple[Check, ...]

class ProjectValidator:
    def __init__(self, root: Path, checks=None)
    def validate(self) -> ProjectReport
```

Roda **todas** as regras sobre o projeto.

```
report = ProjectValidator(Path(".")).validate()

if not report.is_clean():
    print(report)
```

Um `Check` amarra **um scanner às regras do seu contexto** — o agrupamento não é decorativo, é o que evita misturar contextos no mesmo engine.

`project_checks()` devolve os **16** checks registrados, cobrindo 28 das 31 regras:

Check	Regras
`Dependency`	001, 002, 003, 005, 006, 007, 008, 009, 010, 011, 012
`HttpInput`	013, 031
`RouteDeclaration`	016, 017, 018
`PlanetStructure`	019
`SecretLiteral`	020
`ResourceExposure`	021
`ControllerSecurityUsage`	022
`SecretOccurrence`	023
`UnsafeSql`	024
`UploadHandling`	025
`ControllerAuthChecks`	026
`TokenUsage`	027
`ErrorExposure`	028
`EntityBinding`	029
`OutboundRequest`	030
`ModuleClasses`	032

Fora daqui, de propósito:

- **004** — absorvida pela 001, não tem classe própria;
- **014** e **015** — o contexto é montado em tempo de execução pelos pipelines da Atmosphere; não há o que varrer estaticamente ainda.

A diferença em relação ao `RuleEngine`. O engine para na primeira violação; o validador **recolhe todas**. Corrigir uma de cada vez, descobrindo a próxima a cada execução, é um jogo de descobre-e-corrige — o validador mostra tudo de uma vez.

Um erro que **não** seja `SolarException` não é recolhido: seria bug na regra ou no scanner, e precisa aparecer como erro, não virar uma "violação" no relatório.

7.5 `ProjectReport`, `CheckResult` e `Violation`

```
CORRECT = "correct"
VIOLATION = "violation"

@dataclass(frozen=True)
class Violation:
    rule: str
    description: str
    message: str
    location: str = ""

@dataclass(frozen=True)
class CheckResult:
    name: str
    violations: tuple[Violation, ...] = ()

    def is_correct(self) -> bool
    def status(self) -> str

@dataclass(frozen=True)
class ProjectReport:
    results: tuple[CheckResult, ...] = ()
    rules: int = 0

    @property
    def violations(self) -> tuple[Violation, ...]
    def is_clean(self) -> bool
    def status(self) -> str
    def by_rule(self) -> dict[str, tuple[Violation, ...]]
    def rules_violated(self) -> int
    def __str__(self) -> str
```

O relatório guarda o resultado **por verificação**, não só as violações. É o que permite mostrar o status de **todas** as verificações — inclusive as que passaram:

```
SOLAR — verificação arquitetural

Dependency                correct
HttpInput                 correct
PlanetStructure           correct
SecretLiteral             violation
...

resultado: violation — 3 violação(ões) em 1 regra(s)
```

Saber o que foi verificado vale tanto quanto saber o que falhou: é o que mostra que a regra está de fato rodando. Um relatório só de falhas não distingue "passou" de "não foi checado".

7.6 `source_files` — o escopo

`nucleus/rules/scanner/source_files.py`

```
DEFAULT_EXCLUDED_DIRECTORIES = frozenset({
    ".git", ".mypy_cache", ".pytest_cache", ".ruff_cache", ".tox",
    ".venv", "__pycache__", "build", "dist", "node_modules", "venv",
    "examples", "fixtures", "samples", "spec", "specs", "test", "tests",
})

TEST_FILENAMES = frozenset({"conftest.py"})
TEST_FILENAME_PREFIXES = ("test_",)
TEST_FILENAME_SUFFIXES = ("_test.py",)

def source_files(root, excluded_directories=None) -> list[Path]
def belongs_to_project(file, root, excluded_directories=None) -> bool
def is_test_file(name: str) -> bool
def is_inside_excluded(path, root, excluded_directories=None) -> bool
def resolve_exclusions(excluded_directories=None) -> frozenset[str]
```

Todo scanner percorre o projeto por aqui, não por `root.rglob("*.py")`.

Os testes não são verificados

Um teste precisa **justamente quebrar as regras** para validar comportamento e descobrir falhas:

```
# tests/database/test_database.py
import sqlalchemy                    # a ORM direto — proibido no código de produção
```

Cobrar arquitetura de um teste é cobrar do lugar errado: o resultado é a regra ficar vermelha por causa de um `assert`, e a equipe aprender a ignorá-la.

Por isso a exclusão acontece **no scanner**, não na regra: o arquivo de teste nem chega a ser interpretado.

Regra de escopo	Detalhe
por **diretório**, não por nome parecido	`tests.py` continua sendo analisado
nome de teste vale **sempre**	`conftest.py`, `test_*.py`, `*_test.py`, onde quer que estejam
`excluded_directories=set()`	**substitui** o padrão — é como um scanner desliga a exclusão

```
source_files(root, excluded_directories=set())   # varre tudo, inclusive tests/
```

7.7 `RoutingConvention`

```
@dataclass(frozen=True)
class RoutingConvention:
    name: str
    route_decorators: frozenset[str]
    route_registrations: frozenset[str]

def flask_routing_convention() -> RoutingConvention
```

Descreve **como um framework declara rotas** — é o que mantém o scanner desacoplado do Flask.

```
flask_routing_convention()
# decorators:     route, get, post, put, patch, delete, head, options
# registrations:  add_url_rule
```

Suportar outro framework não exige mexer em regra nenhuma: basta fornecer outra convenção.

```
RouteScanner(root, convention=fastapi_routing_convention())
```

7.8 Os scanners

Scanner	Contexto	Responde
`ArchitectureScanner`	`Dependency`	quem depende de quem
`HttpInputScanner`	`HttpInput`	quais entradas HTTP existem e o que declaram
`RouteScanner`	`RouteDeclaration`	como cada rota foi declarada
`PlanetScanner`	`PlanetStructure`	quais Planets existem e o que há na raiz
`SecretScanner`	`SecretLiteral`	valores literais em nomes sensíveis
`ResourceExposureScanner`	`ResourceExposure`	quais campos um Resource expõe
`ControllerSecurityScanner`	`ControllerSecurityUsage`	quais módulos um Controller importa
`ControllerReturnScanner`	`ControllerReturn`	o que cada método de Controller devolve
`SecretContentScanner`	`SecretOccurrence`	segredos pelo **conteúdo**, não pelo nome
`UnsafeSqlScanner`	`UnsafeSql`	SQL montado por concatenação
`UploadHandlingScanner`	`UploadHandling`	primitivos de arquivo dentro de um Controller
`ControllerAuthCheckScanner`	`ControllerAuthChecks`	verificações de autenticação no Controller
`TokenSecurityScanner`	`TokenUsage`	como o token é emitido e lido
`ErrorExposureScanner`	`ErrorExposure`	detalhes internos de erro saindo do handler
`MassAssignmentScanner`	`EntityBinding`	dados da requisição virando Entity/Model
`OutboundRequestScanner`	`OutboundRequest`	como a URL de uma chamada externa é montada
`ModuleClassesScanner`	`ModuleClasses`	quais classes cada arquivo declara

Dois são compartilhados por mais de uma regra: `HttpInputScanner` (013 e 031) e `ArchitectureScanner` (onze regras). O custo de varrer é pago uma vez.

O que é um Controller

Quatro scanners falam de Controller — `ControllerReturnScanner` (014), `ControllerSecurityScanner` (022), `UploadHandlingScanner` (025) e `ControllerAuthCheckScanner` (026). Os quatro compartilham UM critério do que é um Controller (`nucleus/rules/scanner/controller_module.py`):

1. o LUGAR do arquivo — a estrutura que a **019 já exige de todo Planet**: `planets/<planeta>/controllers/<arquivo>.py` ;
2. e, DENTRO dele, o sufixo `Controller` no nome da classe.

Um `controllers/` fora de `planets/` não vale, e um `controller.py` solto também não: sem o Planet, não há Controller. Foi assim que o `orbits/health/controller.py` — que tinha o nome e não o lugar — virou o Planet `solar`, e passou a ser auditado como qualquer outro Controller.

O sufixo do nome sozinho não serve. A própria regra `RoutesMustPointToController` termina em "Controller" e era auditada como Controller pelas três regras que olhavam só o nome — nada nela é um Controller. Ela passava por sorte: não importa nada da lista proibida da 022, não verifica autenticação e não manipula arquivo. Um `import hashlib` a teria transformado em violação de uma regra que não tem relação com ela.

---

8. Nucleus — Banco de dados

`nucleus/database/`

8.1 `Database` — a fachada

```
class Database:
    _driver: DatabaseDriver | None = None
    orm = None
    model = None

    MODELS_PACKAGE = "orbits.database.models"
    SYNC_ON_START = True

    @classmethod
    def init(cls, driver: DatabaseDriver) -> None
    @classmethod
    def driver(cls) -> DatabaseDriver
    @classmethod
    def session(cls)
    @classmethod
    def sync(cls, *, prune=True) -> list[str]
    @classmethod
    def close(cls) -> None
```

`session()` e `sync()` são **repasses**: a fachada não sabe o que é uma sessão nem o que é um schema — ela pergunta ao driver (§8.2). O Planet pede a sessão ao `Database` e nunca importa `sqlalchemy` para obtê-la (SOLAR-ARCH-010).

Mas `init` não é repasse. Ele é o ponto onde o banco se monta:

```
cls._driver.connect()

if cls.SYNC_ON_START:
    cls.sync()
```

E `sync()` faz o que o driver não tem como fazer sozinho — **importar** os Models:

```
cls.driver()                  # recusa antes da montagem, com o motivo
imported = cls._import_models()
cls.driver().sync(prune=prune)
```

Constante	Serve para
`MODELS_PACKAGE`	de onde os Models são importados — um Model novo é um arquivo novo
`SYNC_ON_START`	liga/desliga a montagem do esquema na subida (§2.4)

`_import_models()` varre o pacote com `pkgutil.iter_modules` (um nível, ordem **alfabética** — a ordem do sistema de arquivos muda de máquina para máquina), importa cada módulo e devolve os nomes. Importar é o que **registra** a tabela na ORM: não há registro separado. Só o primeiro nível é varrido, e isso basta porque a regra do projeto é um Model por **arquivo**.

Por que a fachada conhece o pacote de Models. Porque é ela quem monta o banco, e a ORM só conhece o Model que passou pelo import. É a única coisa que o Nucleus sabe sobre o conteúdo do projeto: um **nome de pacote**, e não uma lista de Models. A lista sai da varredura, então um Model novo entra sozinho.

A **única** porta para a infraestrutura de persistência.

```
Database.init(DriverSQLAlchemy(config))

Database.orm    # a infraestrutura ORM, exposta pela fachada
Database.model  # a base de modelagem
```

```
   Orbits ──────► Database (fachada) ──────► Driver ──────► SQLAlchemy
       │                                        │
       └── nunca fala com o driver ─────────────┘
           nem conhece a ORM
```

`init` valida o contrato antes de guardar:

```
if not isinstance(driver, DatabaseDriver):
    raise ContractException(
        "O driver fornecido não implementa o contrato DatabaseDriver.",
        code="INVALID_DATABASE_DRIVER"
    )
```

E `driver()` recusa ser chamado antes de `init`:

```
raise DatabaseException("O Database ainda não foi inicializado.",
                        code="DATABASE_NOT_INITIALIZED")
```

É esse par que faz a diferença entre "a aplicação quebrou em produção porque a ORM não estava pronta" e "a aplicação disse claramente que não foi inicializada".

8.2 `DatabaseDriver`

```
class DatabaseDriver(ABC):
    def connect(self) -> None
    def disconnect(self) -> None
    def health_check(self) -> bool
    def orm(self)
    def model(self)
    def session(self)
    def sync(self, *, prune=True) -> None
```

O contrato da implementação tecnológica. `orm()` e `model()` são o que a fachada reexporta — é por eles que o Planet usa a ORM **sem importá-la**.

`session()` devolve a sessão de trabalho e recusa ser chamado sem conexão:

```
raise DatabaseException("O banco não está conectado.",
                        code="DATABASE_NOT_CONNECTED")
```

`sync()` cria o que falta, altera o que mudou e — com `prune=True`, o **padrão** — remove o que não existe mais no Model. O driver **não sabe quais** tabelas existem: ele sabe o que estiver registrado na base da ORM, e quem importa os Models antes de chamar é a fachada (§8.1).

8.3 `DriverSQLAlchemy`

```
class Base(DeclarativeBase): ...

class DriverSQLAlchemy(DatabaseDriver):
    def __init__(self, config)
    def connect(self) -> None
    def disconnect(self)
    def health_check(self)
    def orm(self)
    def model(self)
    def session(self)
    def sync(self, *, prune=True) -> None
    def _build_connection_url(self)
```

`sync()` faz duas coisas, nesta ordem:

Etapa	Como	O que
1	`Base.metadata.create_all`	cria a tabela que **não existe** — inteira, com índices e chaves
2	Alembic (`produce_migrations`)	altera o que **existe e mudou**: coluna, tipo, obrigatoriedade

A ordem não é gosto: a comparação da etapa 2 é feita **depois** da etapa 1, então o que a etapa 1 criou não aparece mais como diferença.

Ambas trabalham só com os Models que foram **importados** — a base da ORM conhece apenas o que passou pelo import. Quem importa é a fachada, antes de chegar aqui (§8.1).

Por que o Alembic. O SQLAlchemy cria tabela, mas não sabe comparar o banco real com os Models: não existe *schema diff* nele. Quem sabe é o Alembic — o mesmo motor do `alembic revision --autogenerate`. Aqui ele é usado **sem** arquivo de configuração e **sem** pasta de migrations: quem aplica o resultado é este driver.

É a segunda dependência de banco do driver, e mora aqui pelo mesmo motivo que a primeira: o Nucleus é dono da tecnologia, e o driver é o único lugar que a conhece (SOLAR-ARCH-010).

```
REMOVAL_OPERATIONS = (
    DropTableOp, DropColumnOp, DropIndexOp,
    DropConstraintOp, DropTableCommentOp,
)
```

As operações de remoção são listadas **uma a uma** porque no Alembic elas não têm uma base comum: cada uma herda por conta própria, e não existe um ancestral único para um `isinstance`. Uma remoção fora

dessa lista ficaria no banco para sempre, mesmo com `prune` — e `test_a_column_that_left_the_model_leaves_the_bank` é o que trava esse caminho.

O padrão REMOVE. Uma coluna que saiu do Model sai do banco: esquema que só cresce acumula estrutura que ninguém mais declara, e a coluna esquecida continua aceitando escrita de quem não sabe que ela deixou de existir. Quem quiser esquema só aditivo chama `Database.sync(prune=False)`.

⚠️ É justamente por isso que `SYNC_ON_START` deve ficar **desligado em produção** (§2.4): quem remove estrutura sozinho precisa ser ligado de propósito, não por padrão de ambiente.

Os limites, ditos de frente

Limite	Detalhe
mudança de **tipo** em SQLite	o SQLite não tem `ALTER COLUMN`; o Alembic resolveria recriando a tabela (*batch mode*), e isso não está feito. Em Postgres funciona
remoção de coluna em SQLite	só a partir do SQLite 3.35 — e falha se a coluna estiver presa a índice ou constraint
sem revisão	isto **sincroniza**; não versiona. Não há histórico nem volta atrás
custo	a etapa 2 compara a cada subida: rápido, mas cresce com o número de tabelas

A sessão fica no atributo privado `_session`, porque `session` é o **nome do método do contrato** — dois donos para o mesmo nome dariam um `AttributeError` silencioso no dia em que alguém chamasse o método.

O **único** lugar do projeto que pode importar `sqlalchemy` (SOLAR-ARCH-010).

`_build_connection_url` monta a URL a partir da configuração do ambiente. Essa é a razão de o `Environment` devolver os valores **crus**: quem sabe montar a URL é o driver, não o Environment.

8.4 A barreira contra SQL injection

A aplicação usa **ORM**: quem escreve consulta escreve objeto, e a ORM parametriza. O Nucleus não tem classe de execução de SQL — quem executa é o driver, a partir da ORM.

O que a arquitetura entrega é a barreira **estática**: a SOLAR-ARCH-024.

```
# [X] concatenação — o valor entra no texto do SQL
"SELECT * FROM documents WHERE owner_id = " + str(owner_id)
query = f"SELECT * FROM documents WHERE owner_id = {owner_id}"
query = "SELECT * FROM documents WHERE owner_id = %s" % owner_id
query = "SELECT * FROM documents WHERE owner_id = {}".format(owner_id)

# [OK] pela ORM: o valor viaja como parâmetro
session.query(DocumentModel).filter(DocumentModel.owner_id == owner_id)
```

A diferença não é estética: no primeiro caso o valor é **interpretado como SQL**; no segundo, é um dado.

Caminho	Quem garante
consulta pela ORM	a própria ORM, por construção
SQL cru (caso raro)	parâmetro ligado (`:nome`), nunca texto
o que escapar disso	a regra SOLAR-ARCH-024 reprova

> A regra **detecta** o que já está escrito errado. É ela — e não uma classe de conveniência — que sustenta a defesa contra SQL injection na SOLAR.

---

9. Nucleus — Storage

`nucleus/storage/`

9.1 `Storage` — a fachada

```
class Storage:
    @classmethod
    def init(cls, driver: StorageDriver) -> None
    @classmethod
    def save(cls, name: str, content: bytes) -> str
    @classmethod
    def read(cls, name: str) -> bytes
    @classmethod
    def exists(cls, name: str) -> bool
    @classmethod
    def delete(cls, name: str) -> None
    @classmethod
    def _ensure_initialized(cls) -> None
```

```
Storage.init(LocalStorageDriver(root="storage"))

referencia = Storage.save("documents/contrato.pdf", conteudo)
conteudo = Storage.read(referencia)
```

`save` devolve uma **referência**, não um caminho absoluto: quem chamou guarda a referência no banco, e onde o arquivo está de fato é assunto do driver.

Trocar o driver local por S3 não muda uma linha do UseCase — é o mesmo contrato.

9.2 `StorageDriver`

```
class StorageDriver(ABC):
    def save(self, name: str, content: bytes) -> str
    def read(self, name: str) -> bytes
    def exists(self, name: str) -> bool
    def delete(self, name: str) -> None
```

9.3 `LocalStorageDriver`

```
class LocalStorageDriver(StorageDriver):
    def __init__(self, root: str | Path = "storage", create: bool = True)
```

O driver local, e o que ele barra:

Situação	Resultado
`../../etc/passwd`	`StorageException` / `STORAGE_PATH_ESCAPE`
nome vazio	`STORAGE_PATH_ESCAPE` / `INVALID_STORAGE_NAME`
arquivo inexistente	`STORAGE_NOT_FOUND`

O **path traversal** é o ataque a considerar: o nome vem do usuário, e `../../` faria o arquivo sair da pasta de armazenamento. O driver resolve o caminho e confere se ele continua dentro da raiz.

---

10. Nucleus — Upload

`nucleus/upload/`

A estrutura pronta para o upload seguro: política, sanitização, validação e verificação de conteúdo.

10.1 `UploadedFile`

```
@dataclass(frozen=True)
class UploadedFile:
    name: str
    content_type: str
    content: bytes

    @property
    def size(self) -> int
    @property
    def extension(self) -> str
```

10.2 `UploadPolicy`

```
@dataclass(frozen=True)
class UploadPolicy:
    allowed_extensions: frozenset[str]
    allowed_content_types: frozenset[str]
    max_size_bytes: int
    max_files: int
    verify_content: bool = True

    def allows_extension(self, extension: str) -> bool
    def allows_content_type(self, content_type: str) -> bool
    def allows_size(self, size: int) -> bool

    @classmethod
    def documents(cls) -> "UploadPolicy"
    @classmethod
    def images(cls) -> "UploadPolicy"
```

A política **não tem padrão**: quem recebe upload declara o que aceita.

```
policy = UploadPolicy(
    allowed_extensions=frozenset({".pdf", ".docx"}),
    allowed_content_types=frozenset({"application/pdf"}),
    max_size_bytes=10 * 1024 * 1024,
    max_files=3,
)
```

Ou pelos presets: `UploadPolicy.documents()` (pdf/office/txt, 10 MB) e `UploadPolicy.images()` (png/jpg/gif/webp, 5 MB).

10.3 `FileSanitizer`

```
class FileSanitizer(Sanitizer):
    def sanitize(self, payload) -> list[UploadedFile]

def upload_files(payload) -> list[UploadedFile]
def safe_filename(name: str) -> str
```

Limpa o **nome** do arquivo:

Entrada	Saída
`../../etc/passwd`	`passwd`
`relatório<>.pdf`	`relatório.pdf`
nome com 300 caracteres	cortado em 255
nome vazio	`arquivo`

O nome que chega do cliente não é confiável: pode conter caminho, caractere de controle ou tamanho absurdo.

10.4 `FileValidator`

```
class FileValidator(Validator):
    def __init__(self, policy: UploadPolicy)
    def validate(self, payload) -> None
```

Confere **tudo** e acumula os problemas, em vez de parar no primeiro:

Verificação	Recusa
vazio	arquivo sem conteúdo
tamanho	acima de `max_size_bytes`
extensão	fora de `allowed_extensions`
tipo declarado	fora de `allowed_content_types`
quantidade	acima de `max_files`
conteúdo	o conteúdo não corresponde ao tipo declarado

O último é o que pega o caso clássico: um script renomeado para `.pdf`. A mensagem diz qual tipo foi detectado de fato.

Acumular em vez de parar é deliberado: quem está enviando um lote com três arquivos errados precisa saber dos três.

10.5 Verificação de conteúdo

`nucleus/upload/file_signatures.py`

```
SIGNATURES = ...
HEADER_SIZE = 32

def sniff(content: bytes) -> str | None
def canonical(content_type: str) -> str
def is_verifiable(content_type: str) -> bool
def matches(content: bytes, content_type: str) -> bool
```

Reconhece o tipo **real** pelos primeiros bytes do arquivo (*magic bytes*) — não pela extensão, que quem envia escolhe.

```
sniff(b"%PDF-1.7\n...")   # "application/pdf"
sniff(b"GIF89a...")       # "image/gif"
sniff(b"%PDF-1.7\n...")   # mesmo com a extensão .docx
```

Função	Papel
`sniff`	detecta o tipo pelo conteúdo
`canonical`	normaliza (`image/jpg` → `image/jpeg` ; `docx` → `application/zip`)
`is_verifiable`	o tipo declarado tem assinatura conhecida? (`text/plain` não tem)
`matches`	o conteúdo corresponde ao tipo declarado?

A estrutura de `SIGNATURES` é **OR de ANDs**: um tipo casa se **qualquer alternativa** casar, e cada alternativa exige que **todos** os seus trechos apareçam no offset certo.

```
# GIF87a OU GIF89a
("image/gif", (
    ((0, b"GIF87a"),),
    ((0, b"GIF89a"),),
))

# RIFF E WEBP
("image/webp", (
    ((0, b"RIFF"), (8, b"WEBP")),
))
```

Confundir isso (tratar `GIF87a` e `GIF89a` como AND) faz o `sniff` devolver `None` para um dos dois — e foi um teste que pegou.

---

11. Nucleus — Broadcast

`nucleus/broadcast/`

11.1 `Event`

```
class Event(ABC):
    def __init__(self, name: str, data: dict | None = None)
```

11.2 `Broadcast` — a fachada

```
class Broadcast:
    @classmethod
    def init(cls, driver: BroadcastDriver) -> None
    @classmethod
    def publish(cls, event: Event) -> None
    @classmethod
    def subscribe(cls, event_name: str, listener: Callable) -> None
    @classmethod
    def unsubscribe(cls, event_name: str, listener: Callable) -> None
```

```
Broadcast.publish(DocumentCreated(name="document.created", data={"id": 42}))


Broadcast.subscribe("document.created", notify_observers)
```

Mesmo padrão de fachada do `Database` e do `Storage`: um contrato, um driver, uma porta.

11.3 `BroadcastDriver` e `InMemoryBroadcast`

```
class BroadcastDriver(ABC):
    def publish(self, payload: dict) -> None
    def subscribe(self, event_name: str, listener: Callable) -> None
    def unsubscribe(self, event_name: str, listener: Callable) -> None


class InMemoryBroadcast(BroadcastDriver): ...
```

O driver em memória serve para desenvolvimento e testes. Para produção, outro driver implementando o mesmo contrato — sem tocar em quem publica.

---

12. Nucleus — Notificação

`nucleus/notication/notification.py`

```
class Notification:
    def __init__(self)
    def add(self, code: str, message: str, field: str | None = None) -> None
    def has_errors(self) -> bool
    def all(self) -> list
    def clear(self) -> None
```

Acumula os problemas de uma operação para devolver todos de uma vez:

```
notificacao = Notification()

notificacao.add("REQUIRED", "O título é obrigatório.", field="title")
notificacao.add("INVALID", "A data é inválida.", field="due_date")

if notificacao.has_errors():
    return {"errors": notificacao.all()}
```

Três campos por item:

Campo	Serve para
`code`	tratamento automático pelo cliente
`message`	o texto que a pessoa lê
`field`	**qual campo** do formulário marcar

O `field` é o que permite o frontend pintar o campo certo em vez de mostrar um bloco de erros no topo. E `clear()` deixa a instância reutilizável em um laço.

> O diretório se chama `notication` (sem o segundo **i**) — o nome está assim no projeto.

---

13. Nucleus — Environment

`nucleus/environment.py`

```
class Environment:
    SUPPORTED_ENVIRONMENTS = {"DEV", "TEST", "PROD"}

    @staticmethod
    def init()
    @staticmethod
    def __read(environment: str, key: str)
```

Carrega e **normaliza** o `.env`, e devolve a configuração crua:

```
config = Environment.init()

# {
#     "APP_ENV": "DEV",
#     "DB_CONNECTION": "postgresql",
#     "DB_NAME": "solar_dev",
#     "DB_HOST": "localhost",
#     "DB_PORT": "5432",
#     "DB_USER": "...",
#     "DB_PASS": "...",
#     "CORS": [...],
#     "PORT": "5000",
#     "HOST": "0.0.0.0",
# }
```

As variáveis levam o prefixo do ambiente. `__read("DEV", "DB_NAME")` lê `DEV_DB_NAME`:

```
APP_ENV=DEV

DEV_DB_CONNECTION=postgresql
DEV_DB_NAME=solar_dev
DEV_DB_HOST=localhost
DEV_DB_PORT=5432
DEV_DB_USER=solar
DEV_DB_PASS=

PROD_DB_CONNECTION=postgresql
PROD_DB_NAME=solar
...
```

Assim o mesmo `.env` carrega DEV, TEST e PROD sem uma variável sobrescrever a outra — e trocar de ambiente é trocar `APP_ENV`.

Valores vazios são normalizados para `None`: `DEV_DB_PASS=` vira ausente, não string vazia. Sem isso, uma senha vazia passaria por "senha configurada".

Um `APP_ENV` fora dos três suportados levanta na hora:

```
raise SolarException(
    f"Ambiente '{environment}' não suportado.",
    code="INVALID_ENVIRONMENT"
)
```

O que o Environment NÃO faz

Ele **não** monta a URL de conexão, **não** escolhe o dialect e **não** interpreta configuração de banco. Isso é do `DriverSQLAlchemy`. A separação é o que permite trocar Postgres por outro banco sem tocar no carregamento de configuração.

14. As 31 regras arquiteturais

O catálogo completo. Cada linha diz **o que a regra barra** e **o que ela libera**.

Código	Regra	Barra	Libera
001	Planet não depende de Planet	`Planet → Planet`	mesmo Planet, e via `satellites.*`
002	Repository não depende de Database	`Repository → Database`	via Persistence
003	Repository usa Persistence	acesso a dados fora da Persistence	`Repository → Persistence → Database`
004	Planets não se comunicam	*(absorvida pela 001)*	—
005	Model não conhece Persistence	`Model → Persistence`	`Model → Database`
006	Model acessa ORM pelo Database	`Model → sqlalchemy` / drivers	`Model → Database → ORM`
007	Persistence mora em Orbits	`persistence` fora de `orbits.database.persistence`	o caminho canônico
008	Database é a fachada	`Orbits → Nucleus.Database.drivers`	`Orbits → Nucleus.Database`
009	Planet não conhece Driver	`Planet → drivers`	`Planet → Database`
010	Driver é dono da tecnologia ORM	`sqlalchemy` fora de `nucleus.database.drivers`	dentro do driver
011	Planet não conhece infra do Nucleus	`Planet → nucleus.database.*` (exceto fachada)	fachadas arquiteturais
012	Satellites são a fronteira externa	`requests` / `httpx` / `aiohttp` fora de `satellites`	dentro de um Satellite
013	Entrada HTTP passa por Sanitização e Validação	entrada que transporta dados sem sanitizer/validator	entrada sem dados
014	Controller retorna Resource	Controller sem Resource	`Controller → Resource → Response`

Código	Regra	Barra	Libera
015	Contratos nomeados	`dict` / `list` literal na fronteira	`DocumentInput`, mesmo herdando de `dict`
016	Rotas declaradas explicitamente	`@app.route` / `@router.get`	`Blueprint` + `add_url_rule`
017	Rota aponta para Controller	rota para UseCase, Repository, lambda	`Route → Controller`
018	Rota mora em módulo de rotas	rota em `service.py`, `controller.py`	`orbits/routes.py`, `planets/<p>/routes.py`
019	Planet tem estrutura padrão	Planet sem `__init__.py`, `routes.py`, `controllers/`, `entity/`, `repository/`, `resources/`, `usecases/`	arquivos extras
020	Sem credencial hardcoded	`SECRET_KEY = "abc"`	`os.environ[...]`, valor com prefixo `SOLAR_DEFAULT`, `# solar:allow-secret`
021	Resource não expõe segredo	`password`, `password_hash`, `secret_key`, tokens	campos públicos
022	Controller não implementa auth	`hashlib`, `bcrypt`, `jwt`, `hmac` no Controller	`nucleus.security`
023	Sem segredo no código-fonte	chave AWS, chave privada, JWT, `ghp_…`	`os.environ`, exemplo, marcador
024	Persistência sem SQL inseguro	concatenação, f-string, `%`, `.format()`	SQL parametrizado
025	Upload passa pelo Storage	`open`, `shutil`, `mimetypes` no Controller	`Storage.save(...)`
026	Autenticação não se espalha	`current_user`, `abort(401)`, `has_permission` no Controller	middleware + `nucleus.security`
027	Token seguro	`algorithm="none"`, algoritmo implícito, sem `exp`, `verify_signature: False`	algoritmo explícito + `exp`
028	Erro não vaza interno	`str(e)`, `f"{e}"`, `traceback.format_exc()` na resposta	detalhe no log
029	Sem mass assignment	`Entity(**request.json)`, `from_dict(request.json)`	campos explícitos, `**DocumentInput(...)`

Código	Regra	Barra	Libera
030	Sem SSRF	URL de `request.args`, `f"http://{host}/…"`	constante, configuração, `UrlGuard`
032	Um Model por arquivo	arquivo com dois Models	um Model por arquivo

14.1 Dependências (001 a 012)

O bloco que sustenta a direção das camadas. Tudo o que ele olha vem de **uma** varredura: os `import` do projeto.

```
# [X] SOLAR-ARCH-001
# orbits/planets/documents/repository.py
from orbits.planets.users.entity import User                # Planet → Planet

# [X] SOLAR-ARCH-002
# orbits/planets/documents/repository/user_repository.py
from nucleus.database.database import Database              # Repository → Database

# [X] SOLAR-ARCH-005
# orbits/database/models/user_model.py
from orbits.database.persistence.persistence import Persistence   # Model → Persistence
```

A regra **007** tem uma sutileza que vale conhecer: ela olha **segmentos** do caminho, não prefixos de texto.

```
"orbits.database.persistence.users.user_persistency"
#                                          ↑ "persistency", não "persistence"
# → NÃO é a camada de Persistence
```

Detalhe de string que muda o resultado — porque `persistency` é outra palavra.

14.2 Entrada HTTP (013, 029)

A rota declara **para onde** a requisição vai **e quem protege a entrada**:

```
# [X] SOLAR-ARCH-013 — transporta dados e NÃO declara guarda nenhum
# planets/documents/routes.py
documents_routes.add_url_rule(
    "/documents",
    "create_document",
    DocumentController.create,
    methods=["POST"],
)

# [OK] — a rota declara quem limpa e quem exige
# planets/documents/routes.py
documents_routes.add_url_rule(
    "/documents",
    "create_document",
    DocumentController.create,
    methods=["POST"],
    sanitizer=DocumentSanitizer,
    validator=DocumentValidator,
)
```

A ligação é a **própria chamada**: rota que transporta dados e não declara guarda é a que a regra acusa. Não há mapa em outro arquivo para consultar — o que protege a entrada está onde ela foi declarada.

```
# [X] SOLAR-ARCH-029 — mass assignment
document = DocumentEntity(**request.json)

# [OK] — os campos estão declarados
document = DocumentEntity(
    title=request.get("title"),
    owner_id=request.get("owner_id"),
)

# [OK] — o Input filtra; o cliente não escolhe os campos
document = DocumentEntity(**DocumentInput(**request.json))
```

O mass assignment é sutil: a requisição abaixo promove quem chamou a administrador, sem que o código tenha uma linha sobre `is_admin`:

```
POST /documents
{"title": "ok", "owner_id": 99, "is_admin": true}
```

14.3 Rotas (016, 017, 018)

```
# [X] SOLAR-ARCH-016 — decorator
@app.route("/documents", methods=["POST"])
def create_document(): ...

# [OK] — registro explícito
documents_routes.add_url_rule(
    "/documents", "create_document", DocumentController.create,
    methods=["POST"],
)

# [X] SOLAR-ARCH-017 — a rota aponta para o UseCase
documents_routes.add_url_rule("/documents", "create", CreateDocumentUseCase.execute)

# [X] SOLAR-ARCH-018 — a rota mora no service
# orbits/planets/documents/service.py
documents_routes.add_url_rule(...)
```

Por que `add_url_rule` e não o decorator. O decorator esconde a rota: quem lê o Controller não sabe que ele é uma rota, e quem lê a rota não sabe o que ela declara. O registro explícito mantém rota e Controller **no mesmo lugar** — e é legível por um scanner, que é o que permite a 013 e a 018 existirem. O que PROTEGE a entrada não fica aqui: fica na Atmosphere do Planet (§14.2).

14.4 Estrutura (019, 032)

```
# [X] SOLAR-ARCH-019 — repository.py solto, sem routes.py
orbits/planets/user/
├── __init__.py
└── repository.py

# [OK]
orbits/planets/user/
├── __init__.py
├── routes.py
├── atmosphere/
├── controllers/
├── repository/
└── usecases/
```

A `atmosphere/` entra na lista porque é a **fronteira de entrada** do Planet: sem ela, não há onde declarar quem protege cada rota (§14.2). É a única das peças obrigatórias que o Controller não enxerga — ele recebe a requisição já tratada.

Os contratos do fluxo moram **dentro** dessas peças, e não em pasta própria. O de ENTRADA (`controllers/inputs/`) fica junto do Controller, que é quem o monta; o de SAÍDA (`usecases/outputs/`) fica junto do UseCase, que é quem o produz. Quem lê o Controller vê, no mesmo lugar, o que ele recebe; quem lê o UseCase vê o que ele devolve. A 019 exige os sete itens e não proíbe subpasta dentro deles.

```
# [X] SOLAR-ARCH-032 — dois Models no mesmo arquivo
# orbits/database/models/entities.py
class UserModel: ...
class DocumentModel: ...

# [OK] — um arquivo por Model
# orbits/database/models/user_model.py
class UserModel: ...
```

O que conta como Model usa a mesma convenção das regras 005/006: **lugar ou nome**.

Critério	Exemplo
segmento `models` no caminho	`orbits.database.models.*` — **toda** classe ali conta
módulo terminando em `model` / `_model`	`planets.documents.document_model`
classe terminando em `Model`	`UserModel`

14.5 Segredos (020, 021, 023)

Duas regras olham segredo por ângulos diferentes:

	020	023
detecta por	**nome**	**conteúdo**
exemplo	`SECRET_KEY = "abc"`	`CHAVE = "AKIAIOSFODNN7EXAMPLE"`

A 023 é a que pega o caso difícil — o segredo com nome inofensivo. E é a que mais precisa de cuidado contra falso positivo, porque um padrão que casa pode casar em documentação, em exemplo e em teste. Ela tem quatro saídas:

```
# [OK] vem do ambiente
CHAVE = os.environ["AWS_ACCESS_KEY_ID"]

# [OK] exemplo conhecido (a AWS publica essa chave na documentação)
EXEMPLO = "AKIAIOSFODNN7EXAMPLE"

# [OK] documentação não é valor usado pelo programa
"""
    Uso:
        HEADER = "-----BEGIN RSA PRIVATE KEY-----"
"""

# [OK] exceção explícita, visível e revisável no pull request
# solar:allow-secret
INICIO = "-----BEGIN RSA PRIVATE KEY-----"
```

A ordem importa em um ponto: a exceção `# solar:allow-secret` vale na **mesma linha**, dentro do literal de várias linhas, ou em um **comentário dedicado** na linha de cima.

```
# [X] NÃO libera a linha de baixo — a exceção viraria um ignore escondido
OUTRA = 1  # solar:allow-secret
CHAVE = "AKIA2B3C4D5E6F7G8H9J"     # ainda é violação
```

14.6 SQL e arquivos (024, 025)

```
# [X] SOLAR-ARCH-024
query = "SELECT * FROM documents WHERE owner_id = " + str(owner_id)
query = f"SELECT * FROM documents WHERE owner_id = {owner_id}"
query = "SELECT * FROM documents WHERE owner_id = %s" % owner_id
query = "SELECT * FROM documents WHERE owner_id = {}".format(owner_id)

# [OK]
query, params = Query(
    "SELECT * FROM documents WHERE owner_id = :owner"
).with_parameters(owner=owner_id).build()
```

```
# [X] SOLAR-ARCH-025 — o Controller manipulando arquivo
class DocumentController:

    def upload(self, file):
        with open(f"uploads/{file.filename}", "wb") as destino:
            destino.write(file.read())

# [OK] — o Controller delega; o UseCase grava pelo Storage
class DocumentController:

    def upload(self, file):
        return self.use_case.execute(file)

class UploadDocumentUseCase:

    def execute(self, file):
        return Storage.save(file.filename, file.content)
```

14.7 Autenticação e token (022, 026, 027)

```
# [X] SOLAR-ARCH-022 — o Controller implementando autenticação
class UserController:

    def login(self, senha):
        return hashlib.sha256(senha.encode()).hexdigest()

# [OK] — usa o componente da arquitetura
class UserController:

    def login(self, senha):
        return Password.verify(senha, self.repository.hash_of(...))
```

```
# [X] SOLAR-ARCH-026 — autenticação espalhada pelo Controller
class DocumentController:

    def list(self):
        if not current_user.is_authenticated:
            abort(401)
        if not has_permission("list"):
            abort(403)
        return self.use_case.execute()

# [OK] — o Controller só delega
class DocumentController:

    @staticmethod
    def list() -> tuple[dict, int]:

        # Nenhuma verificação de sessão aqui: quem decide quem entra
        # é o middleware, ANTES de o fluxo chegar ao Controller.
        output = list_documents_use_case().execute()

        return DocumentResource.to_object(output), 200
```

```
# [X] SOLAR-ARCH-027
jwt.encode(payload, key, algorithm="none")          # sem assinatura
jwt.encode(payload, key)                            # ou sem algoritmo explícito
jwt.encode({"sub": "1"}, key, algorithm="HS256")    # sem "exp"
jwt.decode(token, key)                              # algoritmo implícito
jwt.decode(token, key, options={"verify_signature": False})

# [OK]
jwt.encode({"sub": "1", "exp": expires_at}, key, algorithm="HS256")
jwt.decode(token, key, algorithms=["HS256"])
```

14.8 Erro e saída (028)

```
# [X] SOLAR-ARCH-028 — o detalhe interno sai na resposta
except Exception as error:
    return {"error": str(error)}
    return jsonify({"erro": f"{error}"})
    raise ValueError(f"falhou: {error}")
    return {"traceback": traceback.format_exc()}

# [OK] — o detalhe vai para o log, a resposta é genérica
except Exception as error:
    logger.exception("falha ao processar o documento")
    return ErrorHandler().handle(error)
```

O que a regra considera "sair" tem três valores, e só o primeiro é violação:

Destino	Significado	Veredito
`response`	demonstrouvelmente sai do handler (`return`, `raise`, resposta montada)	[X] violação
`log`	vai para uma chamada de log	[OK]
`local`	não foi possível provar que sai	[OK]

A distinção existe porque passar **perto** de um `return` não basta: o detalhe precisa estar **no valor retornado**. Sem esse cuidado, guardar a mensagem em um registro de relatório seria acusado — e a regra reprovaria o próprio código que a executa.

14.9 Chamada externa (030)

```
# [X] SOLAR-ARCH-030
requests.get(request.args["url"])
httpx.get(f"http://{host}/internal")
requests.get(url)
requests.get(self.callback_url)

# [OK]
requests.get("https://api.exemplo.com/users")
requests.get(f"{BASE_URL}/users")
requests.get(environment.get("API_URL"))

# [OK] dinâmica, mas validada
UrlGuard(allowed_hosts=["api.exemplo.com"]).validate(url)
requests.get(url)
```

O critério é a **origem** da URL, pela convenção do projeto: constante em `UPPER_CASE` foi decidida no código; nomes como `environment`, `settings` e `config` foram decididos na configuração. Qualquer outro identificador que componha a URL é tratado como não confiável.

---

15. Casos de uso

Cinco receitas completas, do arquivo à verificação.

15.1 Um Planet novo, do zero

Objetivo: criar o Planet `documents` com uma rota que cria documento.

```
orbits/planets/documents/
├── __init__.py
├── routes.py
├── controllers/
│   ├── document_controller.py
│   └── inputs/
│       └── document_input.py
├── repository/
│   └── document_repository.py
└── usecases/
    ├── create_document_usecase.py
    └── outputs/
        └── document_output.py
```

Os contratos nomeados (regra 015)

Cada contrato mora ao lado de quem o usa: o de **entrada** junto do Controller, que é quem o monta; o de **saída** junto do UseCase, que é quem o produz.

```
# controllers/inputs/document_input.py
from dataclasses import dataclass


@dataclass(frozen=True)
class DocumentInput:
    title: str
    owner_id: int
```

```
# usecases/outputs/document_output.py
from dataclasses import dataclass


@dataclass(frozen=True)
class DocumentOutput:
    id: int
    title: str
    owner_id: int
```

O Resource (regras 014 e 021)

```
# orbits/planets/documents/resources/document_resource.py
from nucleus.contracts.resource import Resource


class DocumentResource(Resource):
    """
    Transforma a saída do UseCase no que vai para a resposta.
    """

    @classmethod
    def to_object(cls, data) -> dict:
        return {
            "id": data.id,
            "title": data.title,
            "owner_id": data.owner_id,
        }
```

Repare no que **não** está ali: nenhum campo interno, nenhum `password`, nenhum objeto completo. O Resource é a fronteira.

O UseCase

```
# orbits/planets/documents/usecases/create_document_usecase.py
from nucleus.contracts.use_case import UseCase

from orbits.planets.documents.usecases.outputs.document_output import (
    DocumentOutput,
)



class CreateDocumentUseCase(UseCase):

    def __init__(self, repository):
        self.repository = repository

    @classmethod
    def contract(cls) -> ResponsibilityContract:
        """
        A ENTRADA é declarada pelo NOME: o `DocumentInput` mora no
        Controller, e importá-lo aqui inverteria a direção.
        """

        return ResponsibilityContract(
            owner="CreateDocumentUseCase.execute",
            input_type="DocumentInput",
            output_type=DocumentOutput,
        )

    def execute(self, document_input) -> DocumentOutput:

        saved = self.repository.create(
            title=document_input.title,
            owner_id=document_input.owner_id,
        )

        return DocumentOutput(
            id=saved["id"],
            title=saved["title"],
        )
```

O Repository (regras 002 e 003)

```
# orbits/planets/documents/repository/document_repository.py
from orbits.database.persistence import persistence


class DocumentRepository:

    def create(self, title: str, owner_id: int):
        return persistence.create_document(title=title, owner_id=owner_id)
```

Dois detalhes: o Repository fala com a **Persistence** (não com o `Database`), e os campos são **explícitos** — nunca o dicionário da requisição inteiro (regra 029).

O Controller (regras 014, 022, 025, 026, 028)

```
# orbits/planets/documents/controllers/document_controller.py
from flask import g

from orbits.planets.documents.controllers.inputs.document_input import (
    create_document_input,
)
from orbits.planets.documents.resources.document_resource import (
    DocumentResource,
)
from orbits.planets.documents.usecases.create_document_usecase import (
    create_document_use_case,
)


class DocumentController:

    @staticmethod
    def create() -> tuple[dict, int]:

        # O corpo JÁ TRATADO pela Atmosphere.
        data = g.solar_data

        # O corpo vira CONTRATO nomeado (029).
        document_input = create_document_input(data)

        output = create_document_use_case().execute(document_input)

        return DocumentResource.to_object(output), 201
```

O Controller: **não** verifica autenticação na mão (026), **não** implementa hash (022), **não** toca em arquivo (025), **não** monta erro com detalhe (028) e **não** traduz a saída — quem traduz é o Resource (014).

O Validator e o Sanitizer (regra 013)

```
from nucleus.contracts.sanitizer import Sanitizer
from nucleus.contracts.validator import Validator
from nucleus.exceptions.validation_exception import ValidationException


class DocumentSanitizer(Sanitizer):

    def sanitize(self, payload):
        return {
            chave: valor.strip() if isinstance(valor, str) else valor
            for chave, valor in payload.items()
        }


class DocumentValidator(Validator):

    def validate(self, payload) -> None:
        if not payload.get("title"):
            raise ValidationException(
                "O título é obrigatório.",
                code="INVALID_DOCUMENT"
            )
```

A rota

```
# orbits/planets/documents/routes.py
from nucleus.middleware.solar_blueprint import SolarBlueprint

from orbits.planets.documents.atmosphere.sanitizers.document_sanitizer import (
    DocumentSanitizer,
)
from orbits.planets.documents.atmosphere.validators.document_validator import (
    DocumentValidator,
)
from orbits.planets.documents.controllers.document_controller import (
    DocumentController,
)



document_routes = SolarBlueprint(
    "documents",
    __name__,
    url_prefix="/documents"
)

# A `rule` é o que vem DEPOIS do prefixo: vazia, a rota fica na raiz do
# Planet — `POST /documents`.
document_routes.add_url_rule(
    "",
    "create_document",
    DocumentController.create,
    methods=["POST"],
    sanitizer=DocumentSanitizer,
    validator=DocumentValidator,
)
```

A rota declara **URL → Controller → método** E **quem protege a entrada** — as duas coisas no mesmo lugar, e é isso que a SOLAR-ARCH-013 lê. O `SolarBlueprint` é o que permite declará-las juntas: o `add_url_rule` do Flask não conhece `sanitizer=` nem `validator=` (§5.9).

A rota, argumento por argumento

Os três primeiros argumentos são coisas **diferentes**, e confundi-los é o erro mais comum de quem chega:

```
document_routes.add_url_rule(
    "",                         # rule      → a URL (depois do prefixo)
    "create_document",          # endpoint  → o NOME INTERNO da rota
    DocumentController.create, # view_func → quem responde
    ...
)
```

Argumento	O que é	Quem usa
`""` ou `"/attachments"`	a **URL**, sem o prefixo do Blueprint	o cliente
`"create_document"`	o **endpoint** — o nome interno da rota	o Flask, e a fronteira (§5.9)
`DocumentController.create`	o **alvo** — quem responde	o Flask, ao chamar a view

A URL final é a **soma** das duas partes:

```
POST /documents
     └──────┘
  url_prefix           rule
  (do Blueprint)   +   (do add_url_rule)
```

Daí a convenção do projeto: **o prefixo identifica o PLANET, a rule identifica o RECURSO** — `/examples` + `/users` = `POST /examples/users`.

O endpoint não é a URL. É o nome pelo qual o Flask conhece a rota dentro da aplicação:

```
url_for("example.create_example_user")   # → "/examples/users"
```

Ele é explícito porque três coisas dependem dele:

1. **`url_for`** — monta a URL sem escrevê-la à mão em lugar nenhum;
2. **a chave dos guardas** — o `SolarBlueprint` guarda o par (Sanitizer, Validator) por endpoint, e o mecanismo da fronteira faz a MESMA chave na hora da requisição (§5.9);
3. **as mensagens das regras** — quando a 017 ou a 018 reprovam uma rota, é este nome que identifica qual.

Um nome legível diz o que a rota FAZ quando aparece num log:

```
create  _  document
  |           |
ação        recurso
```

Quando o Planet tem mais de uma rota, o nome do Planet entra no meio — `create_example_user` —, porque é o que distingue o `create` do Planet `example` do `create` de outro Planet numa linha de log.

E quando o nome não é passado? O Flask deriva da FUNÇÃO de visão:

```
document_routes.add_url_rule(
    "",
    DocumentController.create,   # é o SEGUNDO argumento, não o terceiro
    methods=["POST"],
)
```

Aí o endpoint vira `"create"`, o nome do método. Funciona — mas o nome explícito evita que dois métodos de mesmo nome, dentro do MESMO Planet, tentem ocupar o mesmo endpoint (o Flask recusa o segundo registro).

Verificar

```
$ python observatory/solar check
SOLAR — verificação arquitetural

Dependency                correct
HttpInput                 correct
RouteDeclaration          correct
PlanetStructure           correct
...

resultado: correct
```

15.2 Upload de arquivo

O caminho: o arquivo entra pelo Validator, é limpo pelo Sanitizer e gravado pelo Storage.

```
# 1. a rota declara o pipeline
document_routes.add_url_rule(
    "/documents/<int:document_id>/attachments",
    "upload_attachment",
    AttachmentController.upload,
    methods=["POST"],
    sanitizer=FileSanitizer,
    validator=FileValidator,
)
```

```
# 2. o pipeline roda na ordem certa
http_input = HttpInput(
    method="POST",
    name="/documents/1/attachments",
    content_type="multipart/form-data",
    sanitizer=FileSanitizer(),
    validator=FileValidator(UploadPolicy.documents()),
)

payload = RequestPipeline().handle(http_input, request.files)
# → validou e sanitizou; se algo estiver errado, ValueError não passa
```

```
# 3. o UseCase grava pelo Storage — nunca com open()
class UploadAttachmentUseCase(UseCase):

    def __init__(self, policy: UploadPolicy):
        self.policy = policy

    def execute(self, request):

        arquivos = upload_files(request)

        if len(arquivos) > self.policy.max_files:
            raise ValidationException(
                "Muitos arquivos.", code="TOO_MANY_FILES"
            )

        return [
            Storage.save(
                f"documents/{arquivo.name}",
                arquivo.content
            )
            for arquivo in arquivos
        ]
```

O que a arquitetura barra aqui, e por quê:

Tentação	O que acontece	Regra
`open(f"uploads/{file.filename}", "wb")`	path traversal no nome do arquivo	025
confiar só na extensão	script renomeado para `.pdf` passa	13 (verificação de conteúdo)
validar e parar no primeiro erro	quem enviou 3 arquivos errados corrige um por vez	13 (acumula)
devolver `str(error)` se a gravação falhar	caminho interno do servidor na resposta	028

15.3 Login com token e proteção contra brute force

```
class LoginUseCase(UseCase):

    def __init__(self, repository, token_service, guard):
        self.repository = repository
        self.token_service = token_service
        self.guard = guard

    def execute(self, request):

        chave = f"email:{request.email}"

        if self.guard.is_locked(chave):
            raise SecurityException(
                "Muitas tentativas. Tente novamente em instantes.",
                code="TOO_MANY_ATTEMPTS"
            )

        usuario = self.repository.find_by_email(request.email)

        if usuario is None or not Password.verify(request.senha, usuario.senha):
            self.guard.register_failure(chave)
            raise SecurityException("Credenciais inválidas.", code="INVALID_CREDENTIALS")

        self.guard.register_success(chave)      # quem acertou é perdoado

        return self.token_service.create(usuario.id)
```

E o token, pelo componente da arquitetura:

```
token = Token(secret_key=identity.secret_key, expires_hours=8)
jwt_string = token.create(id_user=42)      # HS256 + exp — passa na regra 027
```

Três decisões que a arquitetura força:

- a senha é conferida com `Password.verify`, nunca com `hashlib` no Controller (022);
- a verificação de sessão **não** fica espalhada no Controller (026);
- o erro de credencial é **genérico** — não diz se o e-mail existe (028 por consequência).

15.4 Respondendo erro sem vazar

```
class DocumentController:

    @staticmethod
    def create():

        try:
            output = create_document_use_case().execute(document_input)
            return DocumentResource.to_object(output), 201

        except ValidationException as error:
            logger.warning("entrada inválida", extra={"reference": ...})
            return ErrorHandler().handle(error), 400

        except Exception as error:
            logger.exception("falha ao criar documento")   # detalhe aqui
            return ErrorHandler().handle(error), 500        # resposta genérica
```

```
{
  "code": "INVALID_DOCUMENT",
  "message": "Não foi possível processar a solicitação.",
  "reference": "a3f1c9d2e0b4"
}
```

O cliente recebe um código estável e uma referência. O time pega a referência e encontra a cadeia completa no servidor. **Nada do interior da aplicação atravessou a fronteira.**

O que o cliente **não** recebe:

```
relation "documents" does not exist
Traceback (most recent call last): ...
/home/app/orbits/database/persistence/...
KeyError: 'owner_id'
```

Cada um desses é informação de reconhecimento: revela estrutura, tecnologia e pontos de falha para quem estiver sondando.

15.5 Consulta pela ORM

```
class DocumentRepository:

    def find_open_by_owner(self, owner_id: int):

        return (
            self.persistence.query(DocumentModel)
            .filter(DocumentModel.owner_id == owner_id)
            .filter(DocumentModel.status == "open")
            .order_by(DocumentModel.created_at.desc())
            .all()
        )
```

O valor de `owner_id` viaja como **parâmetro**, nunca colado no texto da consulta.

Se o caso exigir SQL cru, o valor entra ligado a um marcador — nunca interpolado:

```
# [OK] o valor é um parâmetro
session.execute(
    text("SELECT * FROM documents WHERE owner_id = :owner"),
    {"owner": owner_id}
)

# [X] o mesmo valor colado no texto da consulta
session.execute(f"SELECT * FROM documents WHERE owner_id = {owner_id}")
```

A última forma é exatamente o que a SOLAR-ARCH-024 reprova.

15.6 O fluxo completo, ponta a ponta — o Planet `example`

Objetivo: mostrar o caminho inteiro de verdade, do arquivo à resposta HTTP, sem pedaços faltando. O Planet `example` existe no projeto exatamente para isso: ele cria um usuário e passa por **todas** as camadas.

```
orbits/planets/example/
├── __init__.py
├── routes.py
├── atmosphere/
│   ├── __init__.py
│   ├── sanitizers/
│   └── validators/
├── controllers/
│   ├── create_user_controller.py
│   └── inputs/
│       └── create_user_input.py
├── entity/
│   └── user_entity.py
├── repository/
│   └── example_repository.py
├── resources/
│   └── user_resource.py
└── usecases/
    ├── create_user_usecase.py
    └── outputs/
        └── create_user_output.py
```

A rota:

```
blueprint = SolarBlueprint("example", __name__, url_prefix="/examples")

blueprint.add_url_rule(
    "/users",
    "create_example_user",
    CreateUserController.create,
    methods=["POST"],
    sanitizer=CreateUserSanitizer,
    validator=CreateUserValidator,
)
```

`POST /examples/users`. O `/examples` é do Blueprint do Planet; o `/users` é da rota.

O caminho de uma requisição, na ordem em que acontece:

Etapa	Quem	O que decide
1. rota	`routes.py`	qual Controller, e quais guardas a entrada declara
2. entrada	`RequestPipeline`	o corpo tem os campos? o texto vem limpo?
3. contrato	`CreateUserController`	corpo HTTP → `CreateUserInput` (029)
4. negócio	`CreateUserUseCase`	o e-mail tem forma de e-mail? já existe?
5. fronteira do Planet	`ExampleRepository`	traduz "usuário" para a Persistence
6. persistência	`UserPersistency` + `Users`	o INSERT, pela ORM
7. fachada	`Database` → driver	a única porta para o banco
8. saída	`UserResource.to_object`	o que sai — e o que não sai
9. resposta	`UserResource`	o que sai — e o que **não** sai

A requisição

```
curl -X POST http://localhost:5000/examples/users \
     -H "Content-Type: application/json" \
     -d '{"name": "Ana", "email": "ana@exemplo.com", "password": "s3nha-boa"}'
```

```
{
  "id": 1,
  "name": "Ana",
  "email": "ana@exemplo.com",
  "created_at": "2026-10-06T13:31:49.482913+00:00"
}
```

Repare no que **não** está no corpo: `password`, `password_hash`, o Model inteiro. O Resource é a fronteira (§15.1, SOLAR-ARCH-021), e a senha foi para o banco como hash do bcrypt — nunca como texto.

Quando a entrada não serve

```
{ "email": "ana@exemplo.com", "password": "s3nha-boa" }
```

```
{
  "code": "MISSING_FIELDS",
  "message": "Não foi possível processar a solicitação.",
  "reference": "c1a7f3e09b42"
}
```

`code`	Quem recusou	Quando
`INVALID_JSON_BODY`	`JsonValidator`	o corpo não é objeto JSON
`MISSING_FIELDS`	`JsonValidator`	campo obrigatório sem valor
`INVALID_EMAIL`	`CreateUserUseCase`	o e-mail não tem forma de e-mail
`EMAIL_TAKEN`	`CreateUserUseCase`	o e-mail já está em uso

As duas primeiras são **estrutura** e acontecem na entrada. As duas últimas são **negócio** e acontecem no UseCase — o validador não decide se o e-mail é válido, e o UseCase não confere se o campo veio.

O `code` diz o que houve para quem chama a API; o `message` que sai é o genérico do `ErrorHandler` (§5.8). Qual campo faltou fica no log, não na resposta.

Os testes do fluxo

`tests/orbits/test_example_planet.py` sobe a aplicação **de verdade**, em sqlite de memória, e chama a rota **de verdade** — não há mock entre o `POST` e o banco:

Teste	O que ele prova
`..._answers_201_with_the_resource`	o caminho completo termina em 201
`..._does_not_expose_the_password`	o Resource não vaza a credencial
`..._trims_the_value_before_persisting`	o Sanitizer roda de verdade
`..._stored_hashed`	o que está no banco é hash, não a senha
`..._missing_field_answers_400`	a recusa da entrada, sem vazar detalhe
`..._repeated_email_answers_400`	a segunda criação bate no banco
`..._the_planet_declares_its_frontier`	a Atmosphere declara os guardas da entrada
`..._the_route_declares_only_the_essentials`	a rota aponta para o Controller
`..._the_atmosphere_treats_the_body_before_the_controller`	o corpo chega tratado

O `Database` é global. Por isso cada teste encerra o Kernel no `finally`: uma conexão deixada aberta contaminaria o teste seguinte, e o defeito apareceria longe de onde foi criado.

16. Estendendo a SOLAR

16.1 Uma regra nova

O passo a passo, na ordem, com o exemplo de uma regra que exige que todo UseCase tenha um `execute` com contrato nomeado.

1. Descreva o fato — um `dataclass` frozen em `nucleus/rules/scanner/`:

```
# nucleus/rules/scanner/use_case_contract.py
from dataclasses import dataclass



@dataclass(frozen=True)
class UseCaseContract:
    """
    Descreve o contrato declarado por um UseCase.
    """

    owner: str
    input_type: object
    output_type: object
    file: str
    line: int
```

2. Escreva o scanner — `ast`, sem executar código, percorrendo por `source_files`:

```python
# nucleus/rules/scanner/use_case_contract_scanner.py
import ast
from pathlib import Path
from typing import Iterable

from nucleus.rules.scanner.source_files import source_files
from nucleus.rules.scanner.use_case_contract import UseCaseContract


class UseCaseContractScanner:

    def __init__(self, root: Path):
        self.root = root.resolve()

    def scan(self) -> list[UseCaseContract]:
        return [
            contract
            for file in source_files(self.root)
            for contract in self._scan_file(file)
        ]

    def _scan_file(self, file: Path) -> list[UseCaseContract]:
        tree = ast.parse(file.read_text(encoding="utf-8"), filename=str(file))
        relative = file.relative_to(self.root).as_posix()

        contracts = []

        for node in ast.walk(tree):
            if not isinstance(node, ast.ClassDef):
                continue
            if not node.name.endswith("UseCase"):
                continue
            # ... extrai o contrato
            contracts.append(
                UseCaseContract(
                    owner=node.name,
                    input_type=None,
                    output_type=None,
                    file=relative,
                    line=node.lineno,
                )
```

```
            )

        return contracts
```

3. Escreva a regra — a política:

```
# nucleus/rules/architecture/use_cases_must_declare_named_contracts.py
from nucleus.exceptions.solar_exception import SolarException
from nucleus.rules.rule import Rule
from nucleus.rules.scanner.use_case_contract import UseCaseContract


class UseCasesMustDeclareNamedContracts(Rule):

    def code(self) -> str:
        return "SOLAR-ARCH-033"

    def description(self) -> str:
        return "Todo UseCase deve declarar os seus contratos de entrada e saída."

    def validate(self, contract: UseCaseContract) -> None:

        if contract.input_type is not None and contract.output_type is not None:
            return

        raise SolarException(
            f"{self.code()}: o UseCase '{contract.owner}' não declara "
            f"os seus contratos em {contract.file}:{contract.line}."
        )
```

4. Registre a verificação em `nucleus/rules/project_validator.py`:

```
Check("UseCaseContract", UseCaseContractScanner, (
    UseCasesMustDeclareNamedContracts(),
)),
```

5. Escreva os testes — o par que prova a regra e o que impede a regra de virar fumaça:

```
def test_missing_contract_is_rejected():
    with pytest.raises(SolarException):
        UseCasesMustDeclareNamedContracts().validate(_contrato(input_type=None))

def test_declared_contracts_are_allowed():
    UseCasesMustDeclareNamedContracts().validate(_contrato())
```

6. Acrescente a linha no catálogo `docs/rules.md`.

> O teste `test_every_rule_is_registered` descobre todas as classes `Rule` em `nucleus/rules/architecture/` e exige que estejam no registro. **Esquecer o passo 4 faz a suíte falhar** — não é possível criar uma regra que nunca roda.

16.2 Um scanner novo

Um scanner novo só é necessário quando a pergunta é **outra**. Três regras de ouro:

1. **Não execute o código.** Tudo vem de `ast`. Um scanner que importa o módulo passa a depender de o módulo ser importável — e um projeto quebrado não é analisável.
2. **Descreva, não julgue.** Se você está escrevendo um `if` que decide se algo é violação, esse `if` pertence à Rule.
3. **Percorra por `source_files`.** É o que mantém o escopo (testes fora) igual em toda a arquitetura.

E o layout já em uso: um arquivo para o contexto, outro para o scanner.

```
nucleus/rules/scanner/
├── use_case_contract.py           o contexto
└── use_case_contract_scanner.py   o scanner
```

Quando duas regras falam da mesma peça

Se duas regras precisam concordar sobre **o que uma peça é**, o critério mora em um arquivo próprio e as duas o importam. Não duplique o `if`:

```
nucleus/rules/scanner/
├── controller_module.py            o critério COMPARTILHADO
├── controller_return.py            o contexto da 014
├── controller_return_scanner.py    o scanner da 014
└── controller_security_scanner.py  o scanner da 022 (importa o critério)
```

A razão é a de sempre: duas cópias divergem. Antes desse arquivo existir, três scanners reconheciam o Controller pelo sufixo do nome da classe — e a própria regra `RoutesMustPointToController`, que termina em "Controller", passava a ser auditada como Controller por elas.

16.3 Um driver novo

Bancos, armazenamento e broadcast seguem o mesmo padrão de três peças:

```
drivers/interface/<algo>_driver.py    o CONTRATO  (ABC)
drivers/<implementação>.py            o DRIVER    (a tecnologia)
<fachada>.py                          a FACHADA   (`init` + `_ensure_initialized`)
```

Para um storage em nuvem:

```
# nucleus/storage/drivers/s3_storage_driver.py
from nucleus.storage.drivers.interface.storage_driver import StorageDriver


class S3StorageDriver(StorageDriver):

    def __init__(self, bucket: str):
        self.bucket = bucket

    def save(self, name: str, content: bytes) -> str:
        ...
        return f"s3://{self.bucket}/{name}"
```

E a troca não toca em nenhum UseCase:

```
Storage.init(S3StorageDriver(bucket="meus-documentos"))
```

O contrato é o que garante isso. Implemente as quatro operações (`save` , `read` , `exists` , `delete`) e devolva uma **referência** em `save` — não um caminho local.

16.4 Um framework novo

A SOLAR não conhece o Flask: conhece uma `RoutingConvention` .

```
from nucleus.rules.scanner.routing_convention import RoutingConvention


def fastapi_routing_convention() -> RoutingConvention:
    return RoutingConvention(
        name="fastapi",
        route_decorators=frozenset({"get", "post", "put", "patch", "delete"}),
        route_registrations=frozenset({"add_api_route"}),
    )
```

```
RouteScanner(root, convention=fastapi_routing_convention()).scan()
HttpInputScanner(root, convention=fastapi_routing_convention()).scan()
```

Nenhuma regra muda. **Isso é o teste de que a abstração está no lugar certo**: se adicionar um framework exigisse mexer em regra, o acoplamento estaria errado.

16.5 Como comentar o código

O comentário é a documentação de manutenção: quem abre o arquivo daqui a seis meses precisa entender **o que** está sendo feito e **por quê**, sem ter que deduzir isso do código.

Três níveis, e cada um responde a uma pergunta diferente:

Nível	Onde	Responde
Módulo	docstring no topo do arquivo	para que serve o arquivo, e como ele se liga ao resto
Contrato	docstring da classe e do método	o que recebe, o que devolve, o que garante
Linha	comentário acima do bloco	**por que** a linha existe — o que quebraria sem ela

O terceiro é o que costuma faltar, e é o mais valioso. `Database.init(DriverSQLAlchemy(config))` diz o que faz; o comentário explica que a aplicação fala com a **fachada** e nunca com o driver (SOLAR-ARCH-008), e que interpretar a configuração crua é decisão do driver.

O que **não** fazer:

- comentar o óbvio (`# incrementa i`);
- repetir o que o nome já diz (`# o nome do usuário`);
- deixar comentário desatualizado — comentário errado é pior que comentário nenhum.

Quando a decisão vem de uma regra da arquitetura, **cite o código dela** no comentário: `SOLAR-ARCH-014`. É o que liga a linha ao motivo, e o motivo ao teste que o verifica.

---

17. Verificação e auditoria

17.1 O comando

```
python observatory/solar check              # o diretório atual
python observatory/solar check <caminho>    # outro projeto
python observatory/solar check --help       # ajuda do subcomando
python observatory/solar --help             # ajuda geral
```

17.2 A saída

```
SOLAR — verificação arquitetural

Dependency                correct
HttpInput                 correct
RouteDeclaration          correct
PlanetStructure           correct
SecretLiteral             correct
SecretOccurrence          correct
UnsafeSql                 correct
UploadHandling            correct
ControllerAuthChecks      correct
TokenUsage                correct
ErrorExposure             correct
EntityBinding             correct
OutboundRequest           correct
ModuleClasses             correct

resultado: correct
auditoria: observatory/solar/check/check-2026-10-05-174115-correct.md
```

> Esta é a saída **real** do projeto hoje: as 17 verificações passam.

Todas as verificações aparecem, inclusive as que passaram. Saber o que foi verificado vale tanto quanto saber o que falhou: é o que mostra que a regra está de fato rodando.

17.3 Códigos de saída

Código	`resultado`	Significado
`0`	`correct`	nenhuma violação
`1`	`violation`	houve violação
`2`	—	caminho inválido ou comando desconhecido

É o código de saída que faz a esteira **reprovar** o build:

```
python observatory/solar check || exit 1
```

17.4 A auditoria

Toda execução registra um arquivo em `observatory/solar/check/`.

```
observatory/solar/check/
├── check-2026-10-05-155458-violation.md
└── check-2026-10-05-162011-correct.md
```

Por que registrar **sempre**, e não só a violação: o registro da última execução é a garantia de que ela aconteceu e de como terminou. Se só a violação gerasse arquivo, "nenhum arquivo novo" seria ambíguo — tanto pode significar "está tudo certo" quanto "ninguém rodou a verificação".

> Isto vale para o **comando**. A subida do servidor usa a política oposta — ver §17.8.

O nome carrega o momento e o resultado, então a ordenação por nome é a ordenação cronológica: o último arquivo diz, sem precisar abrir, como terminou a última verificação.

Conteúdo:

```
# SOLAR — auditoria da verificação arquitetural

|                |                                          |
| -------------- | ---------------------------------------- |
| resultado      | correct                                  |
| gerado em      | 2026-10-05 17:41:15 (America/Sao_Paulo) |
| raiz verificada | /solar                                  |
| verificações   | 16                                       |
| regras         | 28                                       |
| violações      | 0                                        |
| regras violadas | 0                                       |

## Verificações

| Verificação     | Status  |
| --------------- | ------- |
| Dependency      | correct |
| HttpInput       | correct |
| PlanetStructure | correct |
| SecretLiteral   | correct |

...

> Sem violação, o arquivo termina aqui: a seção de violações só existe quando há o que
> registrar.
```

O horário é o de **São Paulo** (`America/Sao_Paulo`), não o do servidor: o registro é lido por pessoas no Brasil. Duas execuções no mesmo segundo não se sobrescrevem.

17.5 O escopo

Os **testes não são verificados**.

Cobrar arquitetura de um teste é cobrar do lugar errado — quem escreve teste precisa justamente quebrar as regras para validar comportamento e descobrir falhas. O resultado de cobrar seria a regra ficar vermelha por causa de um `assert`, e a equipe aprender a ignorá-la.

Categoria	O que sai
Testes	`tests/`, `test/`, `spec/`, `specs/`, `fixtures/`
Apoio	`examples/`, `samples/`
Nome	`conftest.py`, `test_*.py`, `*_test.py`
Técnicos	`.git/`, `__pycache__/`, `.venv/`, `venv/`, `build/`, `dist/`, `node_modules/`

A exclusão é por **diretório**, não por nome parecido: um `tests.py` continua sendo analisado. E o nome de teste vale em qualquer lugar: um `test_*.py` no meio do código de produção continua sendo teste.

A exclusão acontece **no scanner**: o arquivo de teste nem chega a ser interpretado. Filtrar na regra significaria interpretar o teste para depois descartar — e uma fixture com sintaxe estranha derrubaria a varredura inteira.

17.6 Cobertura

Situação	Regras
Verificadas pelo comando	as 28 com scanner estático
Verificadas em runtime	**014** e **015**, pelos pipelines da Atmosphere
Sem verificação	**004**, absorvida pela 001

17.7 O registro de erros

O Observatório guarda **cada erro tratado** em um banco SQLite próprio — separado do banco da aplicação, e fora do fluxo dela.

```
observatory/
├── records/
│   ├── error_record.py   o que um erro registrado carrega
│   └── error_log.py      o banco e as consultas
├── data/
│   └── solar.sqlite      o banco do Observatório
└── solar/                o comando de verificação
```

A ligação é feita pelo **gancho de observação** do `ErrorHandler`, e a direção da dependência é Observatory → Nucleus — o Nucleus não conhece o Observatório:

```
log = ErrorLog(solar_id=solar_id, environment=environment)

handler = ErrorHandler(observers=[log])

try:
    ...
except Exception as error:
    return handler.handle(error, {
        "method": request.method,
        "route": request.path,
        "status": 500,
    }), 500
```

Cada linha da tabela `error_log` guarda:

Coluna	Conteúdo
`moment`	quando aconteceu (fuso de São Paulo)
`code`	o código que foi para o cliente
`message`	a mensagem **genérica** que foi para o cliente
`reference`	o identificador que liga a resposta ao registro
`exception_type`	a classe da exceção
`detail`	o traceback completo
`method` / `route` / `status`	o contexto da requisição
`solar_id` / `environment`	qual aplicação, e em que ambiente

O detalhe interno fica **aqui**, não na resposta: é a separação que a SOLAR-ARCH-028 exige.

```
log.find("d3adbeef")   # o registro daquela referência
log.count()            # quantos erros desde sempre
log.all()              # todos, do mais antigo ao mais recente
```

> Registrar é **best-effort**: um observador que falha não muda a resposta de quem já recebeu um erro. O Observatório observa — não participa.

17.8 A verificação na subida do servidor

A verificação não é só do comando: **toda subida** do servidor verifica a arquitetura, antes de montar.

```
$ python server.py
verificando a arquitetura...
arquitetura: correct
 * Serving Flask app 'solar'
 * Debug mode: off
 * Running on http://localhost:5000
```

> Saída **real** do projeto hoje.

Quem faz isso é o `Server` (`server.py`), no mesmo ponto onde o Observatório se liga — e pelo mesmo motivo: o Kernel não pode chamar o verificador (ele vive no Nucleus; o verificador, no Observatório), e o Server está fora dos dois.

A política de registro é diferente

Quem roda	Quando	Registra
`python observatory/solar check`	a pessoa pediu	**sempre**
`python server.py`	roda sozinha	**só em violação**

A diferença é QUEM pediu a verificação. No comando, o arquivo é a resposta — e registrar sempre é o que garante que "nenhum arquivo novo" não possa acontecer por execução. Na subida ninguém pediu nada: gravar um arquivo idêntico a cada `python server.py` encheria o diretório de auditoria e afogaria o registro que importa, que é o da violação.

Na prática: subir um projeto `correct` cem vezes **não deixa arquivo nenhum**; subir um projeto com violação deixa um.

Quando há violação

O relatório inteiro vai para a saída — com o local e a mensagem de cada caso — e o arquivo é criado. Exemplo **ilustrativo** (o projeto está `correct` hoje):

```
verificando a arquitetura...
arquitetura: violation

SOLAR — verificação arquitetural

Dependency                correct
HttpInput                 correct
...
SecretLiteral             violation
...

SOLAR-ARCH-020 · Credenciais e segredos não podem estar hardcoded no código. (1)
  config/settings.py:12  SOLAR-ARCH-020: o valor de 'SECRET_KEY' está hardcoded em
  config/settings.py:12. Credenciais e segredos devem vir do Environment, nunca do
  código-fonte.

resultado: violation — 1 violação(ões) em 1 regra(s)
auditoria: observatory/solar/check/check-2026-10-06-121630-violation.md
```

A subida **não** é interrompida: a verificação informa, e quem decide o que fazer com a informação é quem está operando. Para transformar isso em bloqueio, o lugar é `Server.verify_architecture()` — ele devolve o relatório justamente para quem chamou poder decidir.

Custo

A verificação completa leva alguns segundos — cerca de 28s no volume compartilhado deste repositório, algo perto de 1s em disco local. São 17 verificações independentes, e cada uma lê e analisa a árvore inteira do projeto.

É por isso que a subida anuncia `verificando a arquitetura...` antes, com `flush`: sem o aviso, a espera pareceria travamento.

---

18. Índice de classes

Referência rápida: onde cada classe vive e para que serve.

Contratos

Classe	Módulo	Para quê
`Validator`	`nucleus/contracts/validator.py`	validar a entrada
`Sanitizer`	`nucleus/contracts/sanitizer.py`	limpar a entrada
`Resource`	`nucleus/contracts/resource.py`	montar a resposta
`UseCase`	`nucleus/contracts/use_case.py`	executar uma responsabilidade
`ResponsibilityContract`	`nucleus/contracts/responsibility_contract.py`	declarar entrada e saída
`HttpInput`	`nucleus/contracts/http_input.py`	descrever uma entrada HTTP

Middleware

Classe	Módulo	Para quê
`RequestPipeline`	`middleware/request_pipeline.py`	validar e sanitizar na ordem certa
`SecurityPolicy`	`middleware/security_policy.py`	configurar as proteções
`SecurityHeaders`	`middleware/security_headers.py`	cabeçalhos obrigatórios e opcionais
`RateLimiter`	`middleware/rate_limiter.py`	limitar requisições por janela
`BruteForceGuard`	`middleware/brute_force_guard.py`	bloquear após N falhas
`CsrfProtection`	`middleware/csrf_protection.py`	emitir e conferir o token CSRF
`ErrorHandler`	`middleware/error_handler.py`	resposta segura + detalhe no log
`apply`	`middleware/atmosphere.py`	a fronteira de entrada do Planet

Segurança

Classe	Módulo	Para quê
`Password`	`security/password.py`	hash e verificação com bcrypt
`Token`	`security/token.py`	emitir e verificar JWT
`Identity`	`security/identity.py`	`SECRET_KEY` e `SOLAR_ID` da aplicação
`AttemptTracker`	`security/attempt_tracker.py`	tentativas e bloqueios por chave
`UrlGuard`	`security/url_guard.py`	validar URL antes da chamada externa

Regras

Classe	Módulo	Para quê
`Rule`	`rules/rule.py`	o contrato de uma regra
`RuleEngine`	`rules/rule_engine.py`	rodar regras sobre um alvo
`Check`	`rules/project_validator.py`	amarrar scanner e regras
`ProjectValidator`	`rules/project_validator.py`	rodar tudo e recolher as violações
`Violation`	`rules/report.py`	uma violação
`CheckResult`	`rules/report.py`	o resultado de uma verificação
`ProjectReport`	`rules/report.py`	o relatório inteiro

Dados e arquivos

Classe	Módulo	Para quê
`Database`	`database/database.py`	fachada da persistência
`DatabaseDriver`	`database/drivers/interface/database_driver.py`	contrato do driver
`DriverSQLAlchemy`	`database/drivers/driver_sqlalchemy.py`	a única classe que conhece a ORM
`JsonSanitizer`	`security/json_sanitizer.py`	limpar o corpo JSON
`JsonValidator`	`security/json_validator.py`	exigir os campos do corpo JSON
`Database.session`	`database/database.py`	repasse para a sessão do driver
`Database.sync`	`database/database.py`	montar o esquema: criar, alterar, remover
`Database.MODELS_PACKAGE`	`database/database.py`	de onde os Models são importados
`Database.SYNC_ON_START`	`database/database.py`	o botão da montagem do esquema
`DriverSQLAlchemy.sync`	`database/drivers/driver_sqlalchemy.py`	criar e ALTERAR, com o Alembic
`Storage`	`storage/storage.py`	fachada de arquivos
`StorageDriver`	`storage/drivers/interface/storage_driver.py`	contrato do driver
`LocalStorageDriver`	`storage/drivers/local_storage_driver.py`	driver local, barra path traversal
`UploadedFile`	`upload/uploaded_file.py`	o arquivo recebido
`UploadPolicy`	`upload/upload_policy.py`	o que o upload aceita
`FileSanitizer`	`upload/file_sanitizer.py`	limpar o nome do arquivo
`FileValidator`	`upload/file_validator.py`	validar tamanho, tipo e conteúdo

Broadcast, notificação e configuração

Classe	Módulo	Para quê
`Broadcast`	`broadcast/broadcast.py`	fachada de eventos
`BroadcastDriver`	`broadcast/drivers/interface/broadcast_driver.py`	contrato do driver
`InMemoryBroadcast`	`broadcast/drivers/in_memory_broadcast.py`	driver em memória
`Event`	`broadcast/event.py`	o acontecimento publicado
`Notification`	`notication/notification.py`	acumular os erros de uma operação
`Environment`	`environment.py`	carregar e normalizar o `.env`

Exceções

Classe	Módulo
`SolarException`	`exceptions/solar_exception.py`
`ContractException`	`exceptions/contract_exception.py`
`ValidationException`	`exceptions/validation_exception.py`
`DatabaseException`	`exceptions/database_exception.py`
`StorageException`	`exceptions/storage_exception.py`
`SecurityException`	`exceptions/security_exception.py`
`BroadcastException`	`exceptions/broadcast_exception.py`

Observatory

Classe / função	Módulo	Para quê
`main`	`observatory/solar/cli.py`	despachar o subcomando
`run_check`	`observatory/solar/commands/check.py`	rodar a verificação
`record`	`observatory/solar/audit.py`	gravar a auditoria (sempre)
`record_violation`	`observatory/solar/audit.py`	gravar só em violação
`render`	`observatory/solar/audit.py`	montar o Markdown da auditoria
`audit_directory`	`observatory/solar/audit.py`	onde a auditoria fica
`display_path`	`observatory/solar/audit.py`	o caminho como ele é mostrado
`ErrorLog`	`observatory/records/error_log.py`	gravar e consultar os erros
`ErrorRecord`	`observatory/records/error_record.py`	um erro registrado

O que ainda está aberto

Documentar o que não está resolvido faz parte de documentar:

Pendência	Detalhe
024 só olha a Persistence	SQL inseguro fora de um módulo com segmento `persistence` não é flagrado
030 decide por nome	sem análise de fluxo, uma URL "lavada" por várias funções não é rastreável
015 sem scanner	o contexto é o contrato nomeado do UseCase, sem verificação estática
Fronteira só tem guardas	a rota declara Sanitizer e Validator; autenticação, autorização e rate-limit por Planet ainda não têm mecanismo — o lugar existe, a peça não
Rota fora da fronteira	o Controller não recebe dado nenhum, e o único aviso é a SOLAR-ARCH-013 na análise estática: em execução, a ausência do dado pode passar por campo opcional
A verificação custa ~25s	são 16 varreduras independentes da mesma árvore; um cache de AST compartilhado derrubaria isso para ordem de 1s

O `python observatory/solar check` sai **`correct`** no projeto: as 17 verificações passam.

As pendências acima não produzem violação hoje, mas **limitam o que a verificação alcança** — e é isso que precisa estar escrito: uma regra que não cobre um caso não é uma regra que aprova o caso, é uma regra que

não olha para ele.

Histórico de conformidade

Regra	Situação
019	os Planets `auth` e `user` foram reestruturados para a estrutura padrão — conformes
031	deixou de existir como regra: o CSRF é do pipeline, não de cada rota
020	os 3 falsos positivos foram resolvidos pela convenção: valor de default leva o prefixo `SOLAR_DEFAULT`, e valor que não é segredo leva `# solar:allow-secret`
010	deixou de acusar teste quando o escopo excluiu `tests/`
028	o próprio `ProjectValidator` acusado pela regra; a regra foi refinada (destino `local`), não o código
024	a classe `Query` saiu: a aplicação usa ORM, e a barreira é a própria regra